

Umgewandelte Java-Dateien

Package: Aussendienstmitarbeiter

Klasse: Abrechnungsprogramm

```
package Aussendienstmitarbeiter;

import java.util.Scanner;

public class Abrechnungsprogramm {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        int counter=0;
        double gesamtprovision=0.0;
        double gesamtspesen=0.0;
        double gesamtfahrtkosten=0.0;
        double gesamtbetrag=0.0;
        int gesamtstage=0;

        String
output=String.format("\n| "+"=".repeat(50)+"BOHRWERK"+"=".repeat(50)+"|\n");
        output+=String.format("| "+"=".repeat(108)+"|\n");

output+=String.format("| "+"=".repeat(46)+"MONATSABRECHNUNG"+"=".repeat(46)+"|\n");
        output+=String.format("| "+"=".repeat(108)+"|\n");
        while (true) {
            System.out.println("Gab es einen (weiteren) Kundenbesuch?");
            System.out.println("Falls ja, besttigen Sie mit: j oder y oder ja oder
yes");
            System.out.println("Falls Sie beenden wollten, geben Sie: EXIT ein");
            String abfrage=sc.next();
            if
(abfrage.toLowerCase().equals("y")||abfrage.toLowerCase().equals("j")||abfrage.toL
owerCase().equals("yes")||abfrage.toLowerCase().equals("ja")){
                counter++;
            }
            else if (abfrage.toLowerCase().equals("exit")){
                break;
            }
            else {
                System.out.println("Sie haben folgendes unzulssiges eingegeben:
"+abfrage);
                continue;
            }
            System.out.print("Geben Sie Anzahl der Tage ein: ");
            int dauer=sc.nextInt();
```

```

        if (gesamttage +dauer>31) {
            System.out.println("Sie haben etwas falsches
eingegeben!\nUnzulssige Maximaldauer: "+(gesamttage+dauer));
            continue;
        }else {

            System.out.print("Geben Sie die Auftragssumme ein: ");
            double auftrag=sc.nextDouble();
            double provisionNeu=berechneProvision(auftrag);
            gesamtprovision +=provisionNeu;
            output+=String.format("|Provision bei Kunde %7d wegen eines
Auftrags in Hhe von: %18.2f : %22.2f |\n",counter,auftrag,provisionNeu);

            System.out.print("geben Sie die Distanz in vollen km ein: ");
            int distanzNeu=sc.nextInt();
            double fahrtkostenNeu = berechneFahrtkosten(distanzNeu);
            gesamtfahrtkosten+=fahrtkostenNeu;
            output+=String.format("|Fahrtkosten bei Kunde %5d wegen %39d km
Fahrt: %22.2f |\n", counter,distanzNeu,fahrtkostenNeu);

            gesamttage +=dauer;
            double spesenNeu=berechneSpesen(dauer);
            gesamtspesen +=spesenNeu;
            output += String.format("|Spesenkosten bei Kunde %4d wegen %30d
Tagen Arbeitszeit: %22.2f |\n", counter, dauer ,spesenNeu );

        }

        output+=String.format("| "+"=" .repeat(108)+" |\n");
        System.out.println();

    }
    gesamtbetrag=gesamtfahrtkosten+gesamtprovision+gesamtspesen;
    output+=String.format("| "+"=" .repeat(108)+" |\n");
    output+=String.format("|=====Anzahl der Kunden: %80d  |\n",counter);
    output+=String.format("|=====Gesamtarbeitstage: %80d  |\n",gesamttage);
    output+=String.format("|=====Gesamtprovision: %82.2f
|\n",gesamtprovision);
    output+=String.format("|=====Gesamtfahrtkosten: %80.2f
|\n",gesamtfahrtkosten);
    output+=String.format("|=====Gesamtspesen: %85.2f  |\n",gesamtspesen);
    output+=String.format("| "+"=" .repeat(108)+" |\n");
    output+=String.format("| "+"=" .repeat(108)+" |\n");
    output+=String.format("|=====GESAMTBETRAG: %85.2f  |\n", gesamtbetrag);
    output+=String.format("| "+"=" .repeat(108)+" |\n");
    output+=String.format("| "+"=" .repeat(108)+" |\n");

    System.out.printf(output);

    sc.close();

}

```

```

public static double berechneProvision(double auftragssumme) {
    double provision=0.0;

    if (auftragssumme > 10000){
        provision=70+0.005*auftragssumme;
    }
    else if (auftragssumme>0 ) {
        provision=20+0.005*auftragssumme;
    }
    else
        System.out.println("Unzulssiger Wert fr die Auftragssumme!");
    return provision;
}

public static double berechneFahrtkosten(int distanz) {
    double fahrtkosten=0.0;
    fahrtkosten=0.4*distanz;
    return fahrtkosten;
}

public static double berechneSpesen(int arbeitstage) {
    double spesen=0.0;
    spesen=20*arbeitstage;
    return spesen;
}
}

```

Package: Bankanwendung

Klasse: Account

```

package Bankanwendung;

import java.time.LocalDate;
import java.util.Objects;

public class Account {

    private final int accountID;
    private double balance;
    private LocalDate creationDate;
    private Person owner;
    public static int counter=0;

    public Account(LocalDate creationDate, Person owner) {
        this.accountID = counter++;
        // this.balance = balance;
    }
}

```

```

        this.creationDate = creationDate;
        this.owner = owner;
    }
    public int getAccountID() {
        return accountID;
    }
    public double getBalance() {
        return balance;
    }
    public LocalDate getCreationDate() {
        return creationDate;
    }
    public Person getOwner() {
        return owner;
    }

    public void deposit (double amount) {
        if (amount>0)
            balance +=amount;
        else
            System.out.printf("Betrag muss positiv sein! Betrag:
%.2f\n\n",amount);
    }

    public void withdraw(double amount) {
        if(balance<amount && amount>0) {
            System.out.printf("Ihr Guthaben reicht nicht aus fr %.2f .\n"
                + "Maximales Guthaben: %.2f \n\n",amount, balance);
        }
        if (amount>0)
            balance-=amount;
        else
            System.out.printf("Betrag muss positiv sein! Betrag: %.2f
\n\n",amount);
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj)
            return true;
        if (obj == null)
            return false;
        if (getClass() != obj.getClass())
            return false;
        Account other = (Account) obj;
        return accountID == other.accountID
            && Double.doubleToLongBits(balance) ==
Double.doubleToLongBits(other.balance)
            && Objects.equals(creationDate, other.creationDate) &&
Objects.equals(owner, other.owner);
    }
    @Override
    public String toString() {
        return "Account [accountID=" + accountID + ", balance=" + balance + ",

```

```

creationDate=" + creationDate
                + ", owner=" + owner + "];
    }

}

```

Klasse: Bank

```

package Bankanwendung;

import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.List;

public class Bank {

    List<Account> accounts= new ArrayList<Account>();

    public void addAccount(Account newAccount) {
        accounts.add(newAccount);
    }

    public void print() {
        String kontoinformationen="";

        for (Account konto:accounts) {
            kontoinformationen += "Konto ["+konto.getAccountID()+"]: Kontostand =
["+konto.getBalance()+"] , Besitzer = ["
                                +konto.getOwner().getFirstName()+",
["+konto.getOwner().getLastName()+"], Erstellt =\n["
                                +konto.getCreationDate()+"]\n";
        }

        System.out.println(kontoinformationen);
    }

    public boolean transfer(int accountFromID, int accountToID, double amount) {
        if(checkTransfer(accounts.get(accountFromID),accounts.get(accountToID),
amount)) {
            accounts.get(accountFromID).withdraw(amount);
            accounts.get(accountToID).deposit(amount);
            System.out.printf("Transfer gettigt am %s: \nBetrag: %.2f \nAbsender:
%s\nEmpfänger: %s\n\n",
LocalDateTime.now(),amount,accounts.get(accountFromID),accounts.get(accountToID));
            return true;
        }
    }
}

```

```

        return false;
    }

    private boolean checkTransfer(Account accountfrom, Account accountTo, double
amount) {
        if (amount<=0) {
            System.out.printf("Ein negativer Betrag: %.2f  ist nicht erlaubt.\n"
                + "Transfer wurde abgebrochen.\n\n",amount);
            return false;
        }
        if (accountfrom.equals(accountTo)) {
            System.out.printf("Es ist nicht erlaubt, eine berweisung auf sich
selbst zu ttigen.\n"
                + "Transfer wurde abgebrochen. "
                + "\nDas Finanzamt und der Zoll wurde informiert!\n\n");
            return false;
        }
        if (accountfrom.getBalance()<amount) {
            System.out.printf("Das Konto ist fr eine berweisung in Hhe von: %.2f
nicht ausreichend gedeckt."
                + " \nTransfer wurde abgebochen. \nMaximales
Guthaben: %.2f \n\n",
                amount,accountfrom.getBalance());
            System.out.println();
            return false;
        }
        return true;
    }
}

```

Klasse: BankAnwendung

```

package Bankanwendung;

import java.time.LocalDate;

public class BankAnwendung {

    public static void main(String[] args) {

        Person p1= new Person("Guiseppe-Luigi","Salmonelle" , 25);
        Person p2 = new Person("Enzo", "Ennio", 31);
        Person p3 = new Person("Alberto","Alfonso", 34);

        Bank ichLegDichRein = new Bank();

        Account konto1 =new Account(LocalDate.of(2026, 1, 12), p1);
        Account konto2= new Account(LocalDate.of(2014, 12,27), p2);
        Account konto3= new Account(LocalDate.of(2011, 7,13), p3);
    }
}

```

```

        ichLegDichRein.addAccount(konto1);
        ichLegDichRein.addAccount(konto2);
        ichLegDichRein.addAccount(konto3);

        konto1.deposit(7000);
        konto2.deposit(12000);
        konto3.deposit(-200);

        ichLegDichRein.print();

        ichLegDichRein.transfer(1, 0, 30000);
        ichLegDichRein.transfer(1, 0, -200);
        ichLegDichRein.transfer(1, 1, 3);
        ichLegDichRein.transfer(1, 0, 300);
        ichLegDichRein.transfer(1, 2, 5000);

        ichLegDichRein.print();

    }

}

```

Klasse: Person

```

package Bankanwendung;

public class Person {

    private int age;
    private String firstName;
    private String lastName;
    public Person( String firstName, String lastName,int age) {
        this.firstName = firstName;
        this.lastName = lastName;
        this.age = age;
    }
    public int getAge() {
        return age;
    }
    public void setAge(int age) {
        this.age = age;
    }
    public String getFirstName() {
        return firstName;
    }
    public void setFirstName(String firstName) {
        this.firstName = firstName;
    }
}

```

```

    public String getLastName() {
        return lastName;
    }
    public void setLastName(String lastName) {
        this.lastName = lastName;
    }
    @Override
    public String toString() {
        return "Person [age=" + age + ", firstName=" + firstName + ", lastName=" +
lastName + "]";
    }
}

```

Package: Bankanwendung_erweitert

Klasse: Account

```

package Bankanwendung_erweitert;

import java.time.LocalDate;
import java.util.Objects;

public class Account {

    private final int accountID;
    private double balance;
    private LocalDate creationDate;
    private Person owner;
    public static int counter=0;

    public Account(LocalDate creationDate, Person owner) {
        this.accountID = counter++;
        // this.balance = balance;
        this.creationDate = creationDate;
        this.owner = owner;
    }
    public int getAccountID() {
        return accountID;
    }
    public double getBalance() {
        return balance;
    }
    public LocalDate getCreationDate() {
        return creationDate;
    }
    public Person getOwner() {

```



```

        return owner;
    }

    // private boolean checkTransfer(Account accountfrom, Account accountTo, double
    amount) {
    //     if (amount<=0) {
    //         System.out.printf("Ein negativer Betrag: %.2f  ist nicht erlaubt.\n"
    //             + "Transfer wurde abgebrochen.\n\n",amount);
    //         return false;
    //     }
    //     if (accountfrom.equals(accountTo)) {
    //         System.out.printf("Es ist nicht erlaubt, eine berweisung auf sich
    selbst zu ttigen.\n"
    //             + "Transfer wurde abgebrochen. "
    //             + "\nDas Finanzamt und der Zoll wurde informiert!\n\n");
    //         return false;
    //     }
    //     if (accountfrom.getBalance(<amount) {
    //         System.out.printf("Das Konto ist fr eine berweisung in Hhe von: %.2f
    nicht ausreichend gedeckt."
    //             + " \nTransfer wurde abgebochen. \nMaximales
    Guthaben: %.2f \n\n",
    //             amount,accountfrom.getBalance());
    //         System.out.println();
    //         return false;
    //     }
    //     return true;
    // }

    public void deposit (double amount) {
        if (amount <= 0) {
            System.out.printf("Betrag muss positiv sein! %.2f \n\n", amount);
            return;
        }
        balance += amount;
    }

    public void withdraw(double amount) {
        if (amount <= 0) {
            System.out.printf("Betrag muss positiv sein! Betrag: %.2f \n\n",
amount);
            return;
        }

        if (balance < amount) {
            System.out.printf("Ihr Guthaben reicht nicht aus fr %.2f .\nMaximal
zur Verfgung stehender Betrag: %.2f \n\n", amount, balance);
            return;
        }

        balance -= amount;
    }

```

@Override

```

    public boolean equals(Object obj) {
        if (this == obj)
            return true;
        if (obj == null)
            return false;
        if (getClass() != obj.getClass())
            return false;
        Account other = (Account) obj;
        return accountID == other.accountID;
    }
    @Override
    public String toString() {
        return "Account [accountID=" + accountID + ", balance=" + balance + ",
creationDate=" + creationDate
            + ", owner=" + owner + "]";
    }

    public boolean canWithdraw(double amount) {
        return amount > 0 && balance >= amount;
    }

    public boolean canDeposit(double amount) {
        return amount > 0;
    }

    public boolean canReceiveTransferFromSavings() {
        return true;
    }

}

```

Klasse: Bank

```

package Bankanwendung_erweitert;

import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.List;

public class Bank {

    List<Account> accounts= new ArrayList<Account>();

    public void addAccount(Account newAccount) {
        accounts.add(newAccount);
    }

    // public void print() {
    //     String kontoinformationen="";
    //

```

```

//      for (Account konto:accounts) {
//          kontoinformationen += "Konto [" +konto.getAccountID()+"]: Kontostand =
["+konto.getBalance()+"] , Besitzer = ["
//              +konto.getOwner().getFirstName()+",
"+konto.getOwner().getLastName()+"], Erstellt =\n["
//              +konto.getCreationDate()+"]\n";
//      }
//
//
//      System.out.println(kontoinformationen);
//  }

//  public boolean transfer(int accountFromID, int accountToID, double amount) {
//      if(checkTransfer(accounts.get(accountFromID),accounts.get(accountToID),
amount)) {
//          accounts.get(accountFromID).withdraw(amount);
//          accounts.get(accountToID).deposit(amount);
//          System.out.printf("Transfer gettigt am %s: \nBetrag: %.2f \nAbsender:
%s\nEmpfänger: %s\n\n",
//
//          LocalDateTime.now(),amount,accounts.get(accountFromID),accounts.get(accountToID));
//          return true;
//      }
//      return false;
//  }

//  private boolean checkTransfer(Account accountfrom, Account accountTo, double
amount) {
//      if (amount<=0) {
//          System.out.printf("Ein negativer Betrag: %.2f  ist nicht erlaubt.\n"
//              + "Transfer wurde abgebrochen.\n\n",amount);
//          return false;
//      }
//      if (accountfrom.equals(accountTo)) {
//          System.out.printf("Es ist nicht erlaubt, eine berweisung auf sich
selbst zu ttigen.\n"
//              + "Transfer wurde abgebrochen. "
//              + "\nDas Finanzamt und der Zoll wurde informiert!\n\n");
//          return false;
//      }
//      if (accountfrom.getBalance(<amount) {
//          System.out.printf("Das Konto ist fr eine berweisung in Hhe von: %.2f
nicht ausreichend gedeckt."
//              + " \nTransfer wurde abgebochen. \nMaximales
Guthaben: %.2f \n\n",
//              amount,accountfrom.getBalance());
//          System.out.println();
//          return false;
//      }
//      return true;
//  }

```

```

@Override
public String toString() {

```

```

        String kontoinformationen = "";

        for (Account konto : accounts) {
            //          kontoinformationen += "Konto [" + konto.getAccountID() + "]:
            Kontostand = [" + konto.getBalance()
            //          + "]" , Besitzer = [" + konto.getOwner().getFirstName() + ", "
            + konto.getOwner().getLastName()
            //          + "], Erstellt =\n[" + konto.getCreationDate() + "]\n";
            kontoinformationen +=konto.toString()+"\n";
        }

        return kontoinformationen;
    }

    public Account getAccountById(int id) {
        for (Account a : accounts) {
            if (a.getAccountID() == id) {
                return a;
            }
        }
        return null;
    }

    // public boolean transfer(Account from, Account to, double amount) {
    //
    //     if (from == null || to == null) {
    //         System.out.println("Konto nicht gefunden.");
    //         return false;
    //     }
    //
    //     if (amount <= 0) {
    //         System.out.println("Betrag muss positiv sein.");
    //         return false;
    //     }
    //
    //     if (from.equals(to)) {
    //         System.out.println("berweisung auf eigenes Konto nicht erlaubt.");
    //         return false;
    //     }
    //
    //     if (from.getBalance() < amount) {
    //         System.out.println("Nicht genug Guthaben.");
    //         return false;
    //     }
    //
    //     from.withdraw(amount);
    //     to.deposit(amount);
    //
    //     System.out.println("berweisung erfolgreich.");
    //     return true;
    // }

    public boolean transfer(Account from, Account to, double amount) {

```

```

        if (!from.canWithdraw(amount)) {
            System.out.println("Absender darf nicht berweisen.");
            return false;
        }

        if (!to.canDeposit(amount)) {
            System.out.println("Empfänger kann nicht empfangen.");
            return false;
        }
        if (!to.canReceiveTransferFromSavings()) {
            System.out.println("Das Konto wurde schon im letzten Jahrtausend
erffnet!");
            return false;
        }

        from.withdraw(amount);
        to.deposit(amount);

        return true;
    }
}

```

Klasse: BankAnwendung

```

package Bankanwendung_erweitert;

import java.time.LocalDate;
import java.time.format.DateTimeFormatter;

import IO.Input;

public class BankAnwendung {

    static int MIN_ALTER=14;
    static int MAX_ALTER=150;
    static DateTimeFormatter deutschesFormat =
DateTimeFormatter.ofPattern("dd.MM.yyyy");
    static Bank ichLegDichRein = new Bank();
    public static void main(String[] args) {

        Person p1= new Person("Guisepppe-Luigi","Salmonelle" , 25);
        Person p2 = new Person("Enzo", "Ennio", 31);
        Person p3 = new Person("Alberto","Alfonso", 34);
        Person p4 = new Person("Geizi","Sparschwein",43);

        //      Bank ichLegDichRein = new Bank();
    }
}

```

```

    Account konto1 =new Account(LocalDate.of(2026, 1, 12), p1);
    Account konto2= new Account(LocalDate.of(2014, 12,27), p2);
    Account konto3= new Account(LocalDate.of(2011, 7,13), p3);

    CheckingAccount dispokonto= new CheckingAccount(LocalDate.of(2023, 5, 26),
p3, 2000);
    SavingsAccount sparkonto= new SavingsAccount(LocalDate.of(2006, 7, 2),
p2,LocalDate.of(2025, 7, 2));
    SavingsAccount sparkonto2= new SavingsAccount(LocalDate.of(2020, 8, 8),
p1,LocalDate.of(2029, 10, 2));
    SavingsAccount sparkonto3= new SavingsAccount(LocalDate.of(1999, 1, 3),
p3,LocalDate.of(2012, 1, 1));
    SavingsAccount sparkonto4= new SavingsAccount(LocalDate.of(2026, 5, 2),
p4,LocalDate.of(2055, 7, 10));

    ichLegDichRein.addAccount(konto1);
    ichLegDichRein.addAccount(konto2);
    ichLegDichRein.addAccount(konto3);
    ichLegDichRein.addAccount(sparkonto);
    ichLegDichRein.addAccount(sparkonto2);
    ichLegDichRein.addAccount(sparkonto3);
    ichLegDichRein.addAccount(sparkonto4);
    ichLegDichRein.addAccount(dispokonto);


    konto1.deposit(7000);
    konto2.deposit(12000);
//    konto3.deposit(-200);
    konto3.deposit(8200);
    sparkonto.deposit(35000);
    sparkonto2.deposit(7000);
    sparkonto3.deposit(55000);
    sparkonto4.deposit(5000);
    dispokonto.deposit(2550);

    rufeMenueAuf();

//    System.out.println(ichLegDichRein.toString());

//    ichLegDichRein.transfer(1, 0, 30000);
//    ichLegDichRein.transfer(1, 0, -200);
//    ichLegDichRein.transfer(1, 1, 3);
//    ichLegDichRein.transfer(1, 0, 300);
//    ichLegDichRein.transfer(1, 2, 5000);

//    System.out.println(ichLegDichRein.toString());

}

public static void rufeMenueAuf() {
    System.out.println("=".repeat(65));

```

```

        System.out.println("=====BANKMEN=====");
        System.out.println("1.: berweisung");
        System.out.println("2.: Konto erstellen");
        System.out.println("3.: bersicht anzeigen");
        System.out.println(("4.: Abbruch/Ende"));
        System.out.println("=".repeat(65));
        System.out.println();
        waehleAusMenue(Input.readInt("Ihre Auswahl: "));
    }

    public static void waehleAusMenue(int auswahl) {
        switch (auswahl) {
            case 1:
            {
                int fromId = Input.readInt("Von Konto-ID: ");
                int toId = Input.readInt("Zu Konto-ID: ");
                double amount = Input.readDouble("Betrag: ");

                Account from = ichLegDichRein.getAccountById(fromId);
                Account to = ichLegDichRein.getAccountById(toId);

                if (from == null || to == null) {
                    System.out.println("Konto nicht gefunden.");
                    rufeMenueAuf();
                    break;
                }

                boolean success = ichLegDichRein.transfer(from, to, amount);

                if (success) {
                    System.out.println("Transaktion erfolgreich
durchgefahrt.");
                } else {
                    System.out.println("Transaktion fehlgeschlagen.");
                }
                rufeMenueAuf();
                break;
            }
            case 2:
            {
                erstelleKonto();
                rufeMenueAuf();
                break;
            }
            case 3:
            {

                printeUebersicht(ichLegDichRein);
                rufeMenueAuf();
                break;
            }
            case 4:
            {
                System.out.println("Tschs!");
            }
        }
    }

```

```

        break;
    }
    default:
    {
        System.out.println("Fehlerhafte Eingabe: "+auswahl);
        rufeMenueAuf();
    }
}

}

private static void printeUebersicht(Bank bank) {
    System.out.println(bank.toString());
}

public static void erstelleKonto() {

    System.out.println("=".repeat(65));
    System.out.println("====KONTOERSTELLUNG====");
    System.out.println("1.: Girokonto");
    System.out.println("2.: Sparkonto");
    System.out.println("3.: Zurck zum Hauptmen");
    System.out.println("4.: Abbruch");
    System.out.println("=".repeat(65));
    System.out.println();
    int auswahl = Input.readInt("Ihre Auswahl: ");

    switch (auswahl) {

        case 1:
        {
            erstelleGiro();
            break;
        }
        case 2:
        {
            erstelleSpar();
            break;
        }
        case 3:{
            rufeMenueAuf();
            break;
        }
        case 4: {
            break;
        }
        default:
        {
            System.out.println("Fehlerhafte Engabe: "+auswahl);
            erstelleKonto();
        }
    }
}

```



```

    }

    public static CheckingAccount erstelleGiro() {
        boolean isSuccess=false;
        String uebernahme = "n";
        double kreditLimit=0;
        System.out.println("=".repeat(65));
        System.out.println("====Girokontodaten====");
        Person owner=erstelleBesitzer();
        kreditLimit=erstelleKreditlimit();
        uebernahme = Input.readString("Konto erstellen? (J/N): ");
        uebernahme= uebernahme.trim();
        if ((uebernahme.toLowerCase().equals("j") ||
uebernahme.toLowerCase().equals("ja")
        || uebernahme.toLowerCase().equals("y") ||
uebernahme.toLowerCase().equals("yes")))) {
            isSuccess=true;
        }
        if (isSuccess) {
            CheckingAccount konto= new CheckingAccount(LocalDate.now(), owner,
kreditLimit);

            System.out.println("Girokonto erfolgreich erstellt.");
            if (konto != null) {
                ichLegDichRein.addAccount(konto);
            }
            return konto;
        }
        else {
            return null;
        }
    }

}

    public static SavingsAccount erstelleSpar() {
        boolean isSuccess=false;
        String uebernahme = "n";
        LocalDate datum=LocalDate.now();
        System.out.println("=".repeat(65));
        System.out.println("====Sparkontodaten====");
        Person owner=erstelleBesitzer();
        datum=erstelleEndDate();
        uebernahme = Input.readString("Konto erstellen? (J/N): ");
        uebernahme= uebernahme.trim();
        if ((uebernahme.toLowerCase().equals("j") ||
uebernahme.toLowerCase().equals("ja")
        || uebernahme.toLowerCase().equals("y") ||
uebernahme.toLowerCase().equals("yes")))) {
            isSuccess=true;
        }
        if (isSuccess) {
            SavingsAccount konto = new SavingsAccount(LocalDate.now(), owner,
datum);

```

```

        System.out.println("Sparkonto erfolgreich erstellt.");
        if (konto != null) {
            ichLegDichRein.addAccount(konto);
        }
        return konto;
    }

    return null;
}

public static double erstelleKreditlimit() {
    String uebernahme = "n";
    double kreditLimit=0;
    do {
        do {
            kreditLimit=Input.readDouble("Wie hoch ist Dispogrenze? ");
            if (kreditLimit<0) {
                System.out.printf("Fehlerhafte Eingabe: %.2f\n",kreditLimit);
            }
        }while (kreditLimit<0);
        System.out.println("====Kreditlimit====");
        System.out.printf("Kreditlimit: %.2f \n",kreditLimit);
        uebernahme = Input.readString("Wert bernehmen? (J/N): ");
        uebernahme= uebernahme.trim();
    } while (!uebernahme.toLowerCase().equals("j") &&
!uebernahme.toLowerCase().equals("ja")
        && !uebernahme.toLowerCase().equals("y") &&
!uebernahme.toLowerCase().equals("yes"));

    return kreditLimit;
}

public static Person erstelleBesitzer() {
    Person pers = new Person(null, null, 0);
    String vorname = "";
    String nachname = "";
    int alter = 0;
    String uebernahme = "n";

    do {
        do {
            vorname = Input.readString("Geben Sie den Vornamen ein: ");
            if (!isValidName(vorname)) {
                System.out.println("Vorname nicht erlaubt! \nIhre Eingabe: " +
vorname);
            }
        } while (!isValidName(vorname));
        do {
            nachname = Input.readString("Geben Sie den Nachnamen ein: ");
            if (!isValidName(nachname)) {
                System.out.println("Nachname nicht erlaubt! \nIhre Eingabe: "

```

```

+ nachname);
    }
} while (!isValidName(nachname));
do {
    alter = Input.readInt("Geben Sie das Alter ein: ");
    if (alter < 0) {
        System.out.println("Fehlerhafte Eingabe: " + alter);
    }
    if (alter < MIN_ALTER) {
        System.out.println("Eine Kontoerffnung ist erst ab
"+MIN_ALTER+" Jahren erlaubt! \nIhre Eingabe: " + alter);
    }
    if (alter > MAX_ALTER) {
        System.out.println("Eine Kontoerffnung ist ber "+MAX_ALTER+"
Jahren nicht erlaubt! \nIhre Eingabe: " + alter);
    }
} while (alter < MIN_ALTER || alter > MAX_ALTER);

    System.out.println("=====Personen Daten===== ");
    System.out.println("Vorname: " + vorname);
    System.out.println("Nachname: " + nachname);
    System.out.println("Alter: " + alter);
    System.out.println();
    uebernahme = Input.readString("Daten bernehmen? (J/N): ");
    uebernahme= uebernahme.trim();
    } while (!uebernahme.toLowerCase().equals("j") &&
!uebernahme.toLowerCase().equals("ja")
        && !uebernahme.toLowerCase().equals("y") &&
!uebernahme.toLowerCase().equals("yes"));
    pers.setAge(alter);
    pers.setFirstName(vorname);
    pers.setLastName(nachname);
    return pers;
}

    public static LocalDate erstelleEndDate() {
        LocalDate endDate=LocalDate.now();
        do {
            endDate=Input.readDate("Geben Sie das Datum des Sparendes ein: ");
            if(endDate.isBefore(LocalDate.now())){
                System.out.println("Sparende darf nicht in der Vergangenheit
liegen! "+endDate.format(deutschesFormat));
            }
        } while (endDate.isBefore(LocalDate.now()));
        return endDate;
    }

    public static boolean isValidName(String name) {
        return name != null
            && name.matches("[\\p{L} -]+");
    }
}

```

```
}
```

Klasse: CheckingAccount

```
package Bankanwendung_erweitert;

import java.time.LocalDate;

public class CheckingAccount extends Account{

    private double creditLimit;

    public CheckingAccount(LocalDate creationDate, Person owner, double
creditLimit) {
        super(creationDate, owner);
        this.creditLimit = creditLimit;
    }

    @Override
    public String toString() {
        return "CheckingAccount [creditLimit=" + creditLimit + " "+
super.toString()+ "]";
    }

    public boolean transferTo(Account other) {
        if (this.equals(other))
            return false;
        return true;
    }

    public boolean checkLimit(double amount) {
        if (amount <=0) {
            System.out.printf("Der Betrag muss positiv sein: %.2f\nTransfer wurde
abgebochen.\n\n",amount);
            return false;
        }

        if (amount>= getBalance()+creditLimit) {
            System.out.printf("Ihr Kreditrahmen fr %.2f reicht nicht aus!",
amount);
            System.out.printf("Ihr Guthaben in Hhe von: %.2f und eine Dispogrenze
in Hhe von: %.2f \nVerfgbarer Betrag: %.2f \nTransfer wurde
abgebochen.\n\n",getBalance(),creditLimit,getBalance()+creditLimit);
            return false;
        }
    }
}
```

```

        return true;
    }

    @Override
    public boolean canWithdraw(double amount) {
        return amount > 0 && (getBalance() + creditLimit) >= amount;
    }

    @Override
    public boolean canDeposit(double amount) {
        return amount > 0;
    }
}

```

Klasse: Person

```

package Bankanwendung_erweitert;

import java.util.Objects;

public class Person {

    private int age;
    private String firstName;
    private String lastName;
    public Person( String firstName, String lastName,int age) {
        this.firstName = firstName;
        this.lastName = lastName;
        this.age = age;
    }
    public int getAge() {
        return age;
    }
    public void setAge(int age) {
        this.age = age;
    }
    public String getFirstName() {
        return firstName;
    }
    public void setFirstName(String firstName) {
        this.firstName = firstName;
    }
    public String getLastName() {
        return lastName;
    }
    public void setLastName(String lastName) {
        this.lastName = lastName;
    }
    @Override

```

```

    public String toString() {
        return "Person [ " + firstName + " " + lastName + " (" + age + ") ]";
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj)
            return true;
        if (obj == null)
            return false;
        if (getClass() != obj.getClass())
            return false;
        Person other = (Person) obj;
        return Objects.equals(firstName, other.firstName) &&
            Objects.equals(lastName, other.lastName);
    }

}

```

Klasse: SavingsAccount

```

package Bankanwendung_erweitert;

import java.time.LocalDate;
import java.time.format.DateTimeFormatter;

public class SavingsAccount extends Account{

    private LocalDate endDate;
    DateTimeFormatter deutschesFormat = DateTimeFormatter.ofPattern("dd.MM.yyyy");

    public SavingsAccount(LocalDate creationDate, Person owner, LocalDate endDate)
    {
        super(creationDate, owner);
        this.endDate= endDate;
    }

    public boolean isFrozen() {
        return LocalDate.now().isBefore(endDate);
    }

    public boolean canTransfer() {
        return !isFrozen();
    }

    @Override

```

```

    public String toString() {
        return "SavingsAccount [endDate=" + endDate + " " + super.toString() + "];
    }

//
// public boolean transferFrom(Account other, double amount) {
//
//     if (isFrozen()) {
//         System.out.printf("Sparkonto ist bis %s gesperrt!\n",
// endDate.format(deutschesFormat));
//         return false;
//     }
//
//     if (amount <= 0 || amount > getBalance()) {
//         System.out.println("Ungltiger Betrag oder nicht genug Guthaben.");
//         return false;
//     }
//
//     if (!this.getOwner().equals(other.getOwner())) {
//         System.out.printf("Eine berweisung ist nur auf ein Konto mit demselben
// Besitzer mglich! %s\nTransfer wurde abgebrochen.\n\n",getOwner());
//         return false;
//     }
//
//     return true;
// }
//
// public boolean transferTo(Account other) {
//     if (canTransfer()) {
//         if(!getCreationDate().isBefore(LocalDate.of(2000, 1, 1))) {
//             if(this.getOwner().equals(other.getOwner())) {
//                 return true;
//             }
//             else {
//                 System.out.printf("Eine berweisung ist nur auf ein Konto
// mit demselben Besitzer mglich! %s\nTransfer wurde abgebrochen.\n\n",getOwner());
//                 return false;
//             }
//         }
//         else {
//             System.out.printf("Ihr Konto wurde vor dem %s erstellt (%s) und
// ist somit nicht fr berweisungen freigegeben.\n"
// + "Bei Fragen wenden Sie sich an einen
// Bankmitarbeiter.\n\n",LocalDate.of(2000, 1,
// 1).format(deutschesFormat),getCreationDate().format(deutschesFormat));
//             return false;
//         }
//     }
//     else {
//         System.out.printf("Ihr Konto ist noch bis zum %s
// eingefrohren!\n",endDate.format(deutschesFormat));
//         return false;
//     }
// }

```

```
// }

@Override
public boolean canWithdraw(double amount) {

    if (isFrozen()) {
        return false;
    }

    return amount > 0 && getBalance() >= amount;
}

@Override
public boolean canDeposit(double amount) {

    if (isFrozen()) {
        return false;
    }

    return amount > 0;
}
@Override
public boolean canReceiveTransferFromSavings() {
    return !getCreationDate().isBefore(LocalDate.of(2000, 1, 1));
}

}
```

Package: CollatzVermutung

Klasse: CollatzProblem

```
package CollatzVermutung;

import IO.Input;

public class CollatzProblem {

    public static void main(String[] args) {

        int startwert;
        do {
            startwert = Input.readInt("Geben Sie einen positive Startwert ein: ");
            if (startwert < 1)
                System.out.println("Fehlerhafte Einabe: " + startwert);
        } while (startwert < 1);

        System.out.println("===== Startwert: " +
startwert + " =====");
    }
}
```



```

int temp = startwert;
int counter = 1;
System.out.printf("%d, \t", startwert);

while (temp != 1) {
    if (temp % 2 == 0) {
        temp = temp / 2;
    } else {
        temp = 3 * temp + 1;
    }
    counter++;
    if (temp != 1)
        System.out.printf("%d, \t", temp);
    else
        System.out.printf("%d", temp);
    if (counter % 10 == 0)
        System.out.println();
}
System.out.println();
System.out.println("=".repeat(76));
System.out.println(
    "Die Anzahl der Elemente der Collatzfolge beim Startwert: " +
startwert + " betr gt: " + counter);
}

}

```

Package: Comparator

Klasse: Person

```

package Comparator;

import java.time.LocalDate;
//import java.util.Comparator;

public class Person implements Comparable<Person> {

    private int alter;
    private String vorname;
    private String nachname;
    private final LocalDate geburtstag;
    private String augenfarbe;

    public Person(int alter, String vorname, String nachname, LocalDate
geburtstag, String augenfarbe) {
        super();
        this.alter = alter;
        this.vorname = vorname;
        this.nachname = nachname;
    }
}

```

```

        this.geburtstag = geburtstag;
        this.augenfarbe = augenfarbe;
    }

    public int getAlter() {
        return alter;
    }

    public void setAlter(int alter) {
        this.alter = alter;
    }

    public String getVorname() {
        return vorname;
    }

    public void setVorname(String vorname) {
        this.vorname = vorname;
    }

    public String getNachname() {
        return nachname;
    }

    public void setNachname(String nachname) {
        this.nachname = nachname;
    }

    public String getAugenfarbe() {
        return augenfarbe;
    }

    public void setAugenfarbe(String augenfarbe) {
        this.augenfarbe = augenfarbe;
    }

    public LocalDate getGeburtstag() {
        return geburtstag;
    }

    @Override
    public String toString() {
        return "Person [alter=" + alter + ", vorname=" + vorname + ", nachname=" +
nachname + ", geburtstag="
            + geburtstag + ", augenfarbe=" + augenfarbe + "]";
    }

    @Override
    public int compareTo(Person o) {
        // natrliche Vergleichslogik
        int comp = this.nachname.compareTo(o.nachname);
        if (comp != 0) {
            return comp;
        }
    }

```

```

        return this.vorname.compareTo(o.vorname);
    }

//====Das macht man direkt in der Main oder als statische Felder oder als Eigene
Klasse====
// public int compareAlter(Person p1, Person p2) {
//     //alternative Vergleichslogik
//     return Integer.compare(p1.getAlter(), p2.getAlter());
// }
//
//
// public int compareGeburtstag(Person p1, Person p2) {
//     //auch alternative Vergleichslogik
//     int compYear=Integer.compare(p1.geburtstag.getYear(),
p2.geburtstag.getYear());
//     int compMonth=Integer.compare(p1.geburtstag.getMonthValue(),
p2.geburtstag.getMonthValue());
//     if (compYear !=0) {
//         return compYear;
//     }
//     if (compMonth !=0) {
//         return compMonth;
//     }
//     return
Integer.compare(p1.geburtstag.getDayOfMonth(),p2.geburtstag.getDayOfMonth());
// }

}

```

Klasse: Personenverwaltung

```

package Comparator;

import java.time.LocalDate;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Collections;
import java.util.Comparator;
import java.util.List;

public class Personenverwaltung {

    public static void main(String[] args) {

        Person[] personenArray = { new Person(45, "Heinz", "Becker",
LocalDate.of(1980, 12, 13), "blau"),
                                new Person(24, "Bert", "Bertsen", LocalDate.of(2002, 2, 13),
"blau"),
                                new Person(16, "Heinz", "Beckert", LocalDate.of(2010, 2, 13),

```

```

"grn"),
        new Person(45, "Heinzine", "Becker", LocalDate.of(1980, 11, 13),
"braun"),
        new Person(16, "Heinz", "Backwaren", LocalDate.of(2010, 1, 1),
"braun"),
        new Person(33, "Bert", "Becker", LocalDate.of(1993, 4, 4),
"blaugrau") });

    List<Person> personenListe = new ArrayList<>
(Arrays.asList(personenArray));

//      List<Person> personenListe=Arrays.asList(personenArray);// hier ist die
Arrayliste unveränderbar->feste Sicht auf das Array
    personenListe.add(new Person(26, "Mila", "Millenium", LocalDate.of(2000,
1, 1), "blaugrn"));

    Comparator<Person> compareAlter = (p1, p2) ->
Integer.compare(p1.getAlter(), p2.getAlter());

    Comparator<Person> compareGeburtstag = (p1, p2) ->
p1.getGeburtstag().compareTo(p2.getGeburtstag());

    //Comparator mit mehrfacher Sortierung:
    Comparator<Person> compareAugenfarbe =
        Comparator.comparing(Person::getAugenfarbe)
            .thenComparing(Person::getNachname)
            .thenComparing(Person::getVorname);

    System.out.println("=====Arrays=====");
    System.out.println("=====Ungeordnet=====");
    for (Person p : personenArray) {
        System.out.println(p.toString());
    }

    Arrays.sort(personenArray);
    System.out.println("=====natrliche Ordnung mit Comparable=====");
    for (Person p : personenArray) {
        System.out.println(p.toString());
    }

    Arrays.sort(personenArray, compareAlter);
    System.out.println("=====altternative Ordnung mit Compare(Alter)");
    for (Person p : personenArray) {
        System.out.println(p.toString());
    }

    Arrays.sort(personenArray, compareGeburtstag);
    System.out.println("=====alternative Ordnung mitCompare(Geburtstag)");
    for (Person p : personenArray) {
        System.out.println(p.toString());
    }

    Arrays.sort(personenArray, compareAugenfarbe);

```

```

        System.out.println("=====alternative Ordnung mitCompare(Augenfarbe-
>Nachname->Vorname)");
        for (Person p : personenArray) {
            System.out.println(p.toString());
        }

        System.out.println();
        System.out.println("=====ArrayListe=====");

        System.out.println("=====Ungeordnet=====");
        for (Person p : personenListe) {
            System.out.println(p.toString());
        }

        Collections.sort(personenListe); // das ist das gleiche wie:
        personenListe.sort(null);
        System.out.println("=====natrliche Ordnung mit Comparable=====");
        for (Person p : personenListe) {
            System.out.println(p.toString());
        }

        personenListe.sort(compareAlter);
        System.out.println("=====altternative Ordnung mit Compare(Alter)");
        for (Person p : personenListe) {
            System.out.println(p.toString());
        }

        personenListe.sort(compareGeburtstag);
        System.out.println("=====alternative Ordnung mit Compare(Geburtstag)");
        for (Person p : personenListe) {
            System.out.println(p.toString());
        }

        personenListe.sort(Comparator.comparing(Person::getAugenfarbe));
        System.out.println("=====alternative Ordnung mitCompare(Augenfabe)");
        for (Person p : personenListe) {
            System.out.println(p.toString());
        }

        int index = Collections.binarySearch(personenListe, new Person(0, "", "",
        LocalDate.now(), "blaugrn"),
            Comparator.comparing(Person::getAugenfarbe));

        System.out.println(index);

        List < String > namenListe = new ArrayList < >() ;
        namenListe . add ( " Alice " ) ;
        namenListe . add ( " Bob " ) ;
        namenListe . add ( " Alice2 " ) ;
        namenListe . add ( " Bob2 " ) ;

```

```

        namenListe . add ( " Alice3 " ) ;
        namenListe . add ( " Bob3 " ) ;

        // Konvertierung der List in ein Array
        String [] namenArray = namenListe . toArray ( new String [0]) ;
        //das new String steht da, damit Java weiss, dasss der Datentyp String
sein soll

        for (String item: namenArray) {
            System.out.println(item);
        }
        System.out.println("Die Lnge des namenArray ist: "+namenArray.length);
        // namenListe.toArray(new String[namenListe.size()]); wrde mehr sinn
machen und geht auch

    }

    public static String[] fuegeElementArrayHinzu(String[] arr, String element) {
        String[] neuesArray= new String[arr.length+1];
        for (int i=0;i< arr.length;i++) {
            neuesArray[i]=arr[i];
        }
        neuesArray[neuesArray.length-1]=element;
        return neuesArray;
    }

    public static int[] fuegeIntElementHinzu(int[] arr,int element) {
        int[] neuesArray = Arrays.copyOf(arr, arr.length+1);
        neuesArray[arr.length]=element;
        return neuesArray;
    }

    public static int[] fuegeIntegerElementHinzu(int[] arr,int element) {
        int[] neuesArray= new int[arr.length+1];
        System.arraycopy(arr, 0, neuesArray, 0, arr.length);
        //System.arraycopy(quelle, startQuelle, ziel, startZiel, anzahl);
        neuesArray[arr.length]=element;
        return neuesArray;
    }

    public static String[] fuegeHinzu(String[] arr, String element) {
        ArrayList<String> liste = new ArrayList<>(Arrays.asList(arr));
        liste.add(element);
        return liste.toArray(new String[0]);
    }

}

```

Package: Datumsberechnung

Klasse: Datumstescht

```
package Datumsberechnung;

public class Datumstescht {

    public static void main(String[] args) {
        Date d1= new Date();
        Date d2 = new Date(29,2,2001);

        System.out.println(d1);
        System.out.println(d2);

    }

}
```

Package: IMEI

Klasse: IMEI_Pruefziffer_berechnen

```
package IMEI;

public class IMEI_Pruefziffer_berechnen {

    public static void main(String[] args) {
        long imeiOhne = 447806517517251;
        System.out.println("IMEI mit PZ ber long: "+calcIMEIpz(imeiOhne));

        String imeiAlsStringOhne="44780651751725";
        System.out.println("Prfziffer: "+calcIMEIpzWithString(imeiAlsStringOhne));
        System.out.println("Imei mit PZ:
"+imeiAlsStringOhne+calcIMEIpzWithString(imeiAlsStringOhne));

        // System.out.println(berechneQuersumme(1234120));
    }

    public static long calcIMEIpz(long imei) {
        long imeiMitPz=imei*10;
        int gewQuersumme=0;
        long temp=imei;
        while (temp>0) {
            int gewichtung=1+((int)Math.log10(temp))%2;
            gewQuersumme +=berechneQuersumme( (int) (gewichtung*(temp%10)));
            temp /=10;
        }
        //System.out.println(gewQuersumme);
        gewQuersumme %=10;
        int pruefziffer=(10-gewQuersumme)%10;
    }

}
```

```

        imeiMitPz+=pruefziffer;
        System.out.println("Prfziffer ber long:"+pruefziffer);

        return imeiMitPz;
    }

    @SuppressWarnings("static-access")
    public static int calcIMEIpzWithString(String imeiOhneAlsString) {
        int pruefziffer=0;
        int gewichteteQS=0;
        for (int i=0;i< imeiOhneAlsString.length();i++) {

            gewichteteQS+=berechneQuersumme(Integer.parseInt(imeiOhneAlsString.valueOf(imeiOhneAlsString.charAt(i)))*(1+i%2));
        }

        //System.out.println("gewichtete quersumme: "+gewichteteQS);
        gewichteteQS %=10;
        pruefziffer=(10-gewichteteQS)%10;
        return pruefziffer;
    }

    public static int berechneQuersumme(int zahl) {
        int temp=zahl;
        int quersumme=0;
        while (temp>0) {
            quersumme +=temp%10;
            temp/=10;
        }

        return quersumme;
    }
}

```

Package: IO

Klasse: Input

```

package IO;

import java.time.LocalDate;
import java.util.Scanner;

public class Input {

    static Scanner sc= new Scanner(System.in);

    public static int readInt(String message) {
        System.out.print(message);
    }
}

```



```

String eingabe="";
int eingegebeneZahl;
eingabe = sc.next();
try {
    eingegebeneZahl = Integer.parseInt(eingabe);
    return eingegebeneZahl;
}
catch(NumberFormatException e){
    //e.getMessage();
    System.out.println("Sie mssen eine Ganzzahl eingeben!");
    return readInt(message);
}

}

public static double readDouble(String message) {
    System.out.print(message);
    String eingabe="";
    double eingegebeneZahl;
    eingabe = sc.next();
    try {
        eingegebeneZahl = Double.parseDouble(eingabe);
        return eingegebeneZahl;
    }
    catch(NumberFormatException e){
        //e.getMessage();
        System.out.println("Sie mssen eine Zahl eingeben!");
        return readDouble(message);
    }
}

public static String readString(String message) {
    System.out.print(message);
    String eingabe = "";
    eingabe = sc.next();
    return eingabe;
}

public static LocalDate readDate(String message) {
    System.out.println(message);
    int jahr=readInt("Geben Sie das Jahr ein: ");

    int monat;
    do {
        monat=readInt("Geben Sie den Monat ein: ");
        if (monat<1 || monat> 12)
            System.out.println("Fehlerhafte Eingabe fr Monat: "+monat);
    }while (monat<1 || monat> 12);

    boolean istSchaltjahr=false;
    if(jahr%400==0 || jahr%100!=0 && jahr%4==0) {
        istSchaltjahr=true;
    }
}

```

```

    int tag;
    int grenze=0;

    switch (monat) {

    case 1,3,5,7,8,10,12:
        grenze =31;
        break;
    case 4,6,9,11:
        grenze =30;
        break;
    case 2:
        grenze =istSchaltjahr? 29:28;
        break;
    default:
        System.out.println("Ungltige Eingabe");
    }
    do {
        tag=readInt("Geben Sie den Tag ein: ");
        if (tag < 1 || tag > grenze)
            System.out.println("Fehlerhafte Eingabe: "+tag +" im Monat:
"+monat+" und im Jahr: "+jahr);

        }while (tag < 1 || tag > grenze);

        LocalDate date=LocalDate.of(jahr,monat,tag );
        return date;
    }

}

```

Klasse: Input

```

package IO;

import java.time.LocalDate;
import java.util.Scanner;

public class Input {

    static Scanner sc= new Scanner(System.in);

    public static int readInt(String message) {
        System.out.print(message);
        String eingabe="";
        int eingegebeneZahl;
        eingabe = sc.next();
        try {
            eingegebeneZahl = Integer.parseInt(eingabe);

```

```

        return eingegebeneZahl;
    }
    catch(NumberFormatException e){
        //e.getMessage();
        System.out.println("Sie mssen eine Ganzzahl eingeben!");
        return readInt(message);
    }
}

public static double readDouble(String message) {
    System.out.print(message);
    String eingabe="";
    double eingegebeneZahl;
    eingabe = sc.next();
    try {
        eingegebeneZahl = Double.parseDouble(eingabe);
        return eingegebeneZahl;
    }
    catch(NumberFormatException e){
        //e.getMessage();
        System.out.println("Sie mssen eine Zahl eingeben!");
        return readDouble(message);
    }
}

public static String readString(String message) {
    System.out.print(message);
    String eingabe = "";
    eingabe = sc.next();
    return eingabe;
}

public static LocalDate readDate(String message) {
    System.out.println(message);
    int jahr=readInt("Geben Sie das Jahr ein: ");

    int monat;
    do {
        monat=readInt("Geben Sie den Monat ein: ");
        if (monat<1 || monat> 12)
            System.out.println("Fehlerhafte Eingabe fr Monat: "+monat);
    }while (monat<1 || monat> 12);

    boolean istSchaltjahr=false;
    if(jahr%400==0 || jahr%100!=0 && jahr%4==0) {
        istSchaltjahr=true;
    }

    int tag;
    int grenze=0;

    switch (monat) {

```

```

        case 1,3,5,7,8,10,12:
            grenze =31;
            break;
        case 4,6,9,11:
            grenze =30;
            break;
        case 2:
            grenze =istSchaltjahr? 29:28;
            break;
        default:
            System.out.println("Ungltige Eingabe");
    }
    do {
        tag=readInt("Geben Sie den Tag ein: ");
        if (tag < 1 || tag > grenze)
            System.out.println("Fehlerhafte Eingabe: "+tag +" im Monat:
"+monat+" und im Jahr: "+jahr);

        }while (tag < 1 || tag > grenze);

        LocalDate date=LocalDate.of(jahr,monat,tag );
        return date;
    }
}

```

Package: Interfaces_und_Exceptions

Klasse: EmptyException

```

package Interfaces_und_Exceptions;

public class EmptyException extends Exception{

    public EmptyException(String message) {
        super(message);
    }
}

```

Klasse: IList

```

package Interfaces_und_Exceptions;

public interface IList {

    void push(String element);
    String pop() throws EmptyException;
}

```

```

    void clear();
    boolean isEmpty();
    int size();

}

```

Klasse: ListTest

```

package Interfaces_und_Exceptions;

public class ListTest {

    public static void main(String[] args) {

        //  IList liste = new RingBuffer(3);

        IList liste = new Stack();

        liste.push("A");
        liste.push("B");
        liste.push("C");
        liste.push("D");

        try {

            while (!liste.isEmpty()) {
                System.out.println(liste.pop());
            }

        } catch (EmptyException e) {

            System.out.println(e.getMessage());
        }

    }

}

```

Klasse: RingBuffer

```

package Interfaces_und_Exceptions;

public class RingBuffer implements IList {

    private String[] listenelemente;

    private int schreiben;
    private int lesen;

```

```

private int counter;

public RingBuffer(int size) {

    listenelemente = new String[size];

    schreiben = 0;
    lesen = 0;
    counter = 0;
}

public RingBuffer() {
    this(100);
}

@Override
public void push(String element) {

    if (counter == listenelemente.length) {

        listenelemente[schreiben] = element;

        schreiben = (schreiben + 1) % listenelemente.length;

        lesen = (++lesen) % listenelemente.length;

        return;
    }

    listenelemente[schreiben] = element;

    schreiben = (schreiben + 1) % listenelemente.length;

    counter++;
}

@Override
public String pop() throws EmptyException {

    if (isEmpty()) {
        throw new EmptyException("RingBuffer ist leer!");
    }

    String geloeschtesElement = listenelemente[lesen];

    listenelemente[lesen] = null;

    lesen = (++lesen) % listenelemente.length;

    counter--;

    return geloeschtesElement;
}

```

```

@Override
public void clear() {

    for (int i = 0; i < listenelemente.length; i++) {
        listenelemente[i] = null;
    }

    schreiben = 0;
    lesen = 0;
    counter = 0;
}

@Override
public boolean isEmpty() {
    return counter == 0;
}

@Override
public int size() {
    return counter;
}
}package Timer;

import javax.swing.*;
import java.awt.*;
import java.awt.event.MouseAdapter;
import java.awt.event.MouseEvent;
import java.awt.geom.Ellipse2D;

public class RoundButton extends JButton{

    private Shape shape;

    public RoundButton(String text) {
        super(text);
        setContentAreaFilled(false);
        setFocusPainted(false);
        setBorderPainted(false);
        setOpaque(false);
        setForeground(Color.WHITE);
        setFont(new Font("SansSerif", Font.BOLD, 14));

        addMouseListener(new MouseAdapter() {
            @Override
            public void mouseEntered(MouseEvent e) {
                setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));
            }
        });
    }

    @Override
    protected void paintComponent(Graphics g) {
        Graphics2D g2 = (Graphics2D) g.create();

```

```

        g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
RenderingHints.VALUE_ANTIALIAS_ON);

        if (getModel().isArmed()) {
            g2.setColor(getBackground().darker());
        } else if (getModel().isRollover()) {
            g2.setColor(getBackground().brighter());
        } else {
            g2.setColor(getBackground());
        }

        g2.fillOval(0, 0, getWidth(), getHeight());
        super.paintComponent(g);
        g2.dispose();
    }

    @Override
    public boolean contains(int x, int y) {
        if (shape == null || !shape.getBounds().equals(getBounds())) {
            shape = new Ellipse2D.Float(0, 0, getWidth(), getHeight());
        }
        return shape.contains(x, y);
    }
}

```

Klasse: Stack

```

package Interfaces_und_Exceptions;

public class Stack implements IList {

    // private int isize;
    private String[] listenelemente;
    private int letzte;

    public Stack(int isize) {
        listenelemente = new String[isize];
        letzte=0;
    }

    public Stack() {
        // this.isize=100;
        this(100);
    }

    private void resize(int neueGroesse) {
        String[] neu = new String[neueGroesse];

        for (int i = 0; i < letzte; i++) {

```



```

        neu[i] = listenelemente[i];
    }

    listenelemente = neu;
}

@Override
public void push(String element) {
    if(letzte==listenelemente.length) {
        resize(listenelemente.length*2);
    }

    listenelemente[letzte]=element;
    letzte++;
}

@Override
public String pop() throws EmptyException {
    if (isEmpty()){
        throw new EmptyException("Stack ist leer!");
    }
    letzte--;
    String geloeschtesElement=listenelemente[letzte];
    listenelemente[letzte]=null;
    if(letzte>0 &&letzte<= listenelemente.length/3) {
        resize(listenelemente.length / 2);
    }

    return geloeschtesElement;
}

@Override
public void clear() {
    for (int i=0;i< listenelemente.length;i++) {
        listenelemente[i]=null;
    }
    letzte=0;
}

@Override
public boolean isEmpty() {
    return letzte==0;
}

@Override
public int size() {
    return letzte;
}

```

```
}  
  
}
```

Package: Kreditrechner

Klasse: Kreditrechner

```
package Kreditrechner;  
  
import IO.Input;  
  
public class Kreditrechner {  
  
    public static void main(String[] args) {  
        double kredit;  
        do {  
            kredit = Input.readDouble("Geben Sie die Kreditsumme ein: ");  
            if (kredit <= 0)  
                System.out.println("Ungltige Eingabe fr eine Kreditsumme: " +  
kredit);  
        } while (kredit <= 0);  
  
        double zins;  
        do {  
            zins = Input.readDouble("Geben Sie den Zinssatz in Prozent pro Jahr  
ein: ");  
            if (zins < 0)  
                System.out.println("Ungltige Eingabe fr den Zinssatz: " + zins +  
"%");  
        } while (zins < 0);  
        zins/=100;  
  
        double rate;  
        do {  
            rate=Input.readDouble("Geben Sie die monatliche Rate ein: ");  
            if(12*rate<=zins*kredit) {  
                System.out.println("Unglte Eingabe fr die monatliche Rate:  
"+rate);  
                System.out.printf("Der Mindestbetrag fr die monatliche Rate  
betrgt: %.2f \n",+Math.ceil((zins*kredit/12.0+0.01)*100)/100.0);  
            }  
  
        }while (12*rate<=zins*kredit);  
  
        double restschuldAnfang=kredit;  
        double restschuldEnde=restschuldAnfang;  
        double tilgung=0.0;  
        double zinsen=0;  
        double gesamtzinsen = 0;
```

```

        int counter=0;
        System.out.println("| Nummer des Monats | Restschuld Monatsanfang |
Anteil Zinsen | Anteil Tilgung | Restschuld Monatsende |");
        while(restschuldEnde*(1-zins/12.0)>rate) {
            restschuldAnfang=restschuldEnde;
            zinsen=Math.round(zins*restschuldAnfang/12.0*100)/100.0;
            gesamtzinsen +=zinsen;
            tilgung=rate-zinsen;
            restschuldEnde=restschuldAnfang-tilgung;
            System.out.printf("|%18d |%25.2f |%14.2f |%15.2f |%23.2f |\n",
(1+counter++),restschuldAnfang,zinsen,tilgung,restschuldEnde);
            if (counter%12==0)
                System.out.println("|-----|-----|-----|
-|-----|-----|-----|");
        }

        restschuldAnfang=restschuldEnde;
        zinsen=Math.round(zins*restschuldAnfang/12.0*100)/100.0;
        gesamtzinsen +=zinsen;
        tilgung=zinsen+restschuldAnfang;
        restschuldEnde=restschuldAnfang+zinsen-tilgung;
        System.out.printf("|%18d |%25.2f |%14.2f |%15.2f |%23.2f |\n",
(1+counter++),restschuldAnfang,zinsen,tilgung,restschuldEnde);
        int jahre=counter/12;
        int monate=counter%12;

        System.out.println("|-----|-----|-----|
-----|-----|-----|");
        System.out.printf("Die Rckzahlung des Kredits dauerte %5d Jahre und %d
Monate.\nInsgesamt wurden %.2f zurckbezahlt.",jahre,monate,
(kredit+gesamtzinsen));

    }
}

```

Package: LIFO_FIFO

Klasse: FIFO_Queue

```

package LIFO_FIFO;

import Interfaces_und_Exceptions.EmptyException;

```

```

import Interfaces_und_Exceptions.IList;

public class FIFO_Queue implements IList {

    String[] arr;
    int counter = 0;

    public FIFO_Queue(int isize) {
        arr = new String[isize];
    }

    public FIFO_Queue() {
        this(100);
    }

    @Override
    public void push(String element) {
        if (counter == arr.length) {
            arr = resize(2 * arr.length);
        }
        arr[counter] = element;
        counter++;
    }

    @Override
    public String pop() throws EmptyException {
        if (counter == 0) {
            throw new EmptyException("Die Liste ist leer.Sie knnen kein Element
von einer leeren Liste lschen!");
        }
        String element = arr[0];
        // String[] neuesArray= new String[arr.length];
        System.arraycopy(arr,1, arr, 0,counter-1);
        counter--;
        arr[counter]=null;
        if (3 * counter <= arr.length) {
            arr = resize(Math.max(arr.length / 2,1));
        }
        return element;
    }

    @Override
    public void clear() {
        for (int i = 0; i < arr.length; i++) { // hier knnte ich auch counter
statt arr.length verwenden
            arr[i] = null;
        }
        counter = 0;
    }

    @Override
    public boolean isEmpty() {
        return counter == 0;
    }
}

```

```

@Override
public int size() {
    return counter;
}

private String[] resize(int newSize) {
    String[] neuesArray = new String[newSize];
    for (int i = 0; i < counter; i++) {
        neuesArray[i] = arr[i];
    }
    return neuesArray;
}

public void zeigeInhalt() {
    for (int i = 0; i < arr.length; i++) {
        System.out.println(arr[i]);
    }
}
}

```

Klasse: FIFO_Ring

```

package LIFO_FIFO;

public class FIFO_Ring implements IList {

    String[] arr;
    int counter=0;
    int leseZeiger=0;
    int schreibZeiger=0;

    public FIFO_Ring(int isize) {
        arr=new String[isize];
    }

    public FIFO_Ring() {
        this(100);
    }

    @Override
    public void push(String element) {
        if(counter==arr.length) {
            try {
                pop();
                // System.out.println("ich habe gerade was gelscht!");
            } catch (EmptyException e) {
                e.printStackTrace();
            }
        }
    }
}

```

```

        }
    }
    arr[schreibZeiger]=element;
    schreibZeiger=(++schreibZeiger) % arr.length;
    counter++;

}

@Override
public String pop() throws EmptyException {
    if(size()==0) {
        throw new EmptyException("Die Liste ist leer.Sie knnen kein Element
von einer leeren Liste lschen!");
    }
    String element=arr[leseZeiger];
    arr[leseZeiger]=null;
    leseZeiger=(++leseZeiger) % arr.length;
    counter--;
    return element;
}

@Override
public void clear() {
    for (int i=0;i<arr.length;i++) {
        arr[i]=null;
    }
    leseZeiger=0;
    schreibZeiger=0;
    counter=0;
}

@Override
public boolean isEmpty() {
    return counter==0;
}

@Override
public int size() {
    return counter;
}

public int berechneRest() {
    if((schreibZeiger-leseZeiger)%arr.length<0) {
        return arr.length + (schreibZeiger-leseZeiger)%arr.length;
    }
    return (schreibZeiger-leseZeiger)%arr.length;
}

public void zeigeInhalt(){
    for (int i=0;i< arr.length;i++) {
        System.out.println(arr[i]);
    }
}
}

```

```
}
```

Klasse: IList

```
package LIFO_FIFO;

public interface IList {

    void push(String element);

    String pop() throws EmptyException;

    void clear();

    boolean isEmpty();

    int size();

}
```

Klasse: LIFO_Stack

```
package LIFO_FIFO;

public class LIFO_Stack implements IList {

    int counter=0;
    String[] arr;

    public LIFO_Stack(int isize) {
        arr=new String[isize];
    }

    public LIFO_Stack() {
        this(100);
    }

    @Override
    public void push(String element) {
        if (counter==arr.length) {
            arr=resize(2*arr.length);
        }
        arr[counter]=element;
        counter++;
    }

    @Override
    public String pop() throws EmptyException{
```

```

        if(size()==0) {
            throw new EmptyException("Die Liste ist leer.Sie knnen kein Element
von einer leeren Liste lschen!");
        }
        String element=arr[counter-1];
        arr[counter-1]=null;
        counter--;
        if(3*counter<= arr.length) {
            arr=resize(Math.min(arr.length / 2,1));
        }
        return element;
    }

    @Override
    public void clear() {
        for (int i=0;i<arr.length;i++) { //hier knnte ich auch counter statt
arr.length verwenden
            arr[i]=null;
        }
        counter=0;
    }

    @Override
    public boolean isEmpty() {
        return counter==0;
    }

    @Override
    public int size() {
        return counter;
    }

    private String[] resize(int newSize) {
        String[] neuesArray=new String[newSize];
        for (int i=0;i<counter; i++) {
            neuesArray[i]=arr[i];
        }
        return neuesArray;
    }

    public void zeigeInhalt(){
        for (int i=0;i< arr.length;i++) {
            System.out.println(arr[i]);
        }
    }

}

```

Klasse: ListTest

```
package LIFO_FIFO;
```



```

public class ListTest {

    public static void main(String[] args) throws EmptyException {

//        LIFO_Stack list= new LIFO_Stack(12);

//        FIFO_Ring list= new FIFO_Ring(12);

        FIFO_Queue list = new FIFO_Queue(12);

        System.out.println("Liste leer: "+list.isEmpty());

        list.push("erstes Element");
        list.push("zweites Element");
        list.push("drittes Element");
        list.push("viertes Element");
        list.push("fnftes Element");
        list.push("sechstes Element");
        list.push("siebtes Element");
        list.push("achtes Element");
        list.push("neuntes Element");
        list.push("zehntes Element");
        list.push("elftes Element");
        list.push("zwlftes Element");
        list.push("dreizehntes Element");
        list.push("vierzehntes Element");

        list.zeigeInhalt();

        try {
            System.out.println("gelschtes Element: "+list.pop());
        } catch (Interfaces_und_Exceptions.EmptyException e) {
            // TODO Auto-generated catch block
            e.printStackTrace();
        }
        list.zeigeInhalt();

        System.out.println("Gre der Liste: "+list.size());

        System.out.println("Liste leer: "+list.isEmpty());

        System.out.println("Liste wird geleert!");
        list.clear();
        list.zeigeInhalt();
        System.out.println("Liste leer: "+list.isEmpty());

//        System.out.println("gelschtes Element: "+list.pop());

    }

}

```

Package: Lebenstage

Klasse: Lebenstage

```
package Lebenstage;

import java.time.LocalDate;
import java.time.format.DateTimeFormatter;

import IO.Input;

public class Lebenstage {

    public static void main(String[] args) {
        LocalDate heute=LocalDate.now();
        DateTimeFormatter deutschesFormat =
        DateTimeFormatter.ofPattern("dd.MM.yyyy");

        LocalDate geburtstag;
        do{
            geburtstag=Input.readDate("Geben Sie das Geburtsdatum ein: ");
            if(geburtstag.isAfter(heute))
                System.out.println("Ungltige Eingabe:
"+geburtstag.format(deutschesFormat));
        }while(geburtstag.isAfter(heute));

        LocalDate zieldatum;
        long tage= geburtstag.datesUntil(heute).count();
        long tageSeitGeburt;
        do{
            tageSeitGeburt=Input.readInt("Geben Sie die gewünschten Tage seit Ihres
Geburtstags ein: ");
            if(tageSeitGeburt<0)
                System.out.println("Fehlerhafte Eingabe: "+tageSeitGeburt);
        }while (tageSeitGeburt<0);

        if (tage!=1)
            System.out.printf("Seit dem %s sind %d Tage
vergangen.\n",geburtstag.format(deutschesFormat),tage);
        else
            System.out.printf("Seit dem %s ist %d Tag
vergangen.\n",geburtstag.format(deutschesFormat),tage);

        zieldatum=geburtstag.plusDays(tageSeitGeburt);
        if (zieldatum.isBefore(heute)) {
            System.out.printf("%d Tage nach Ihrem Geburtstag war der:
%s",tageSeitGeburt,zieldatum.format(deutschesFormat));
        }
        else {
            System.out.printf("%d Tage nach Ihrem Geburtstag wird der: %s
```

```

sein.\n",tageSeitGeburt,zieldatum.format(deutschesFormat));
    }

}

}

```

Package: Lottosimulation

Klasse: Lottosimulation

```

package Lottosimulation;

import IO.Input;

public class Lottosimulation {

    public static void main(String[] args) {
        int[] tippschein=readUserTip();

        int n;
        do {
            n=Input.readInt("Mindestens wieviel richtige Zahlen wollen Sie zu
Ihrem Lottotipp haben? ");
            if(n<1||n>6) {
                System.out.println("Fehlerhafte Eingabe: "+n);
            }
        } while (n<1||n>6);

        int counter=1;

        while ( checkTip(tippschein, drawNumbers())<n) {
            counter++;
        }

        System.out.printf("Um %d richtige Zahlen mit Ihrem Tipp zu erhalten
benötigten Sie %d Spiele.\n",n,counter);

    }

    public static int[] drawNumbers() {
        int[] arr= new int[50];
        for (int i=0;i<6;i++) {
            int gezogeneZahl=(int)(49*Math.random()+1);
            if (arr[gezogeneZahl]!=1) {
                arr[gezogeneZahl]=1;
            }else {
                while (arr[gezogeneZahl]==1) {
                    gezogeneZahl=(int)(49*Math.random()+1);
                }
            }
        }
    }
}

```

```

        arr[gezogeneZahl]=1;
    }
}
return arr;
}

public static int[] readUserTip() {
    int[] arr=new int[50];
    int tip=0;
    for (int i=1;i<=6;i++) {
        do {
            tip=Input.readInt("Geben Sie Ihre "+i+".te Zahl ein: ");
            if (tip <1 || tip>49) {
                System.out.println("Ungltige Eingabe: "+tip);
            }
        } while(tip <1 || tip>49);

        if (arr[tip]!=1) {
            arr[tip]=1;
        }
        else{
            while (arr[tip]==1) {
                System.out.println("Diese Zahl hatten Sie schon!!!");
                System.out.println("Wollen Sie gar nicht gewinnen?");
                do {
                    tip=Input.readInt("Geben Sie Ihre "+i+".te Zahl ein: ");
                    if (tip <1 || tip>49) {
                        System.out.println("Ungltige Eingabe: "+tip);
                    }
                } while(tip <1 || tip>49);
            }
            arr[tip]=1;
        }
    }

    return arr;
}

public static int checkTip(int[] tip, int[]lotteryNumbers) {
    int treffer=0;
    for(int i=0;i<50;i++) {
        treffer+=tip[i]*lotteryNumbers[i];
    }
    return treffer;
}

}

```

Klasse: Lottosimulation2

```

package Lottosimulation;

import IO.Input;

public class Lottosimulation2 {

    public static void main(String[] args) {
        int[] tippschein = readUserTip();

        int n;
        do {
            n = Input.readInt("Mindestens wieviel richtige Zahlen wollen Sie zu  
Ihrem Lottotipp haben? ");
            if (n < 1 || n > 6) {
                System.out.println("Fehlerhafte Eingabe: " + n);
            }
        } while (n < 1 || n > 6);

        int counter = 1;
        int[] richtige = new int[6];
        int [] temp =drawNumbers();
        while (checkTip(tippschein, temp)[0] < n) {
            temp =drawNumbers();
            counter++;
        }
        for (int i=1;i<7;i++) {
            richtige[i-1]= checkTip(tippschein, temp)[i];
        }

        System.out.printf("Um %d richtige Zahlen mit Ihrem Tipp zu erhalten  
benötigten Sie %d Spiele.\n", n, counter);
        System.out.println("Die 6 Richtigen waren:");
        for (int i=0;i< richtige.length;i++)
            System.out.printf("Die %d. gezogene Zahl war: %2d\n", (i+1),
richtige[i]);

    }

    public static int[] drawNumbers() {
        int[] arr = new int[56];
        for (int i = 0; i < 6; i++) {
            int gezogeneZahl = (int) (49 * Math.random() + 1);
            if (arr[gezogeneZahl] != 1) {
                arr[gezogeneZahl] = 1;
                arr[50 + i] = gezogeneZahl;
            } else {
                while (arr[gezogeneZahl] == 1) {
                    gezogeneZahl = (int) (49 * Math.random() + 1);
                }
                arr[gezogeneZahl] = 1;
            }
        }
    }
}

```

```

        arr[50 + i] = gezogeneZahl;
    }
}
return arr;
}

public static int[] readUserTip() {
    int[] arr = new int[50];
    int tip = 0;
    for (int i = 1; i <= 6; i++) {
        do {
            tip = Input.readInt("Geben Sie Ihre " + i + ".te Zahl ein: ");
            if (tip < 1 || tip > 49) {
                System.out.println("Ungltige Eingabe: " + tip);
            }
        } while (tip < 1 || tip > 49);

        if (arr[tip] != 1) {
            arr[tip] = 1;
        } else {
            while (arr[tip] == 1) {
                System.out.println("Diese Zahl hatten Sie schon!!!");
                System.out.println("Wollen Sie gar nicht gewinnen?");
                do {
                    tip = Input.readInt("Geben Sie Ihre " + i + ".te Zahl ein: ");

                    if (tip < 1 || tip > 49) {
                        System.out.println("Ungltige Eingabe: " + tip);
                    }
                } while (tip < 1 || tip > 49);
            }
            arr[tip] = 1;
        }
    }

    return arr;
}

public static int[] checkTip(int[] tip, int[] lotteryNumbers) {
    int treffer = 0;
    int[] ausgabe = new int[7];
    for (int i = 0; i < 50; i++) {
        treffer += tip[i] * lotteryNumbers[i];
    }
    ausgabe[0] = treffer;
    for (int i = 1; i < 7; i++) {
        ausgabe[i] = lotteryNumbers[49 + i];
    }

    return ausgabe;
}
}

```

Package: Matritzenrechner

Klasse: Matritzenrechner_mit_verbotenen_Spur

```
package Matritzenrechner;

import IO.Input;

public class Matritzenrechner_mit_verbotenen_Spur {

    public static void main(String[] args) {

        int zeilenanzahl;
        do {
            zeilenanzahl= Input.readInt("Geben Sie die Anzahl der Zeilen ein: ");
            if(zeilenanzahl<1) {
                System.out.println("Fehlerhafte Eingabe: "+zeilenanzahl);
            }
        } while (zeilenanzahl<1);

        int spaltenanzahl;
        do {
            spaltenanzahl= Input.readInt("Geben Sie die Anzahl der Spalten an: ");
            if (spaltenanzahl<1) {
                System.out.println("Fehlerhafte Eingabe: "+ spaltenanzahl);
            }
        } while (spaltenanzahl<1);

        int[][] originalmatrix = erzeugeMatrix(zeilenanzahl, spaltenanzahl);

        System.out.println("Welche Operation wollen Sie durchfhren?");
        System.out.println("T:  Transponieren");
        System.out.println("Z:  Zeilensumme");
        System.out.println("S:  Spaltensumme");
        System.out.println("P:  Spurberechnung");
        String operation=Input.readString("Whllen Sie eine der 4 Operationen: ");

        System.out.println("Orginalmatzrix:");
        zeigeMatrix(originalmatrix);
        treffeAuswahl(operation, originalmatrix);

    }

    public static int[][] transponieren(int[][] a) {
        int[][] transponierte = new int[a.length][a[0].length];
        for (int i = 0; i < a.length; i++) {
```

```

        for (int j = 0; j < a[0].length; j++) {
            transponierte[i][j] = a[j][i];
        }
    }
    return transponierte;
}

public static int[] zeilensumme(int[][] a) {
    int[] zeilensumme = new int[a.length];
    for (int i = 0; i < a.length; i++) {
        for (int j = 0; j < a[0].length; j++) {
            zeilensumme[i] += a[i][j];
        }
    }
    return zeilensumme;
}

public static int[] spaltensumme(int[][] a) {
    int[] spaltensumme = new int[a[0].length];
    for (int j = 0; j < a[0].length; j++) {
        for (int i = 0; i < a.length; i++) {
            spaltensumme[j] += a[i][j];
        }
    }
    return spaltensumme;
}

public static int spur(int[][] a) {
    int spur = 0;
    int diagonalelemente = Math.min(a.length, a[0].length);
    for (int i = 0; i < diagonalelemente; i++) {
        spur += a[i][i];
    }
    return spur;
}

public static int[][] erzeugeMatrix(int zeilen, int spalten) {
    int[][] erzeugte = new int[zeilen][spalten];
    for (int i = 0; i < zeilen; i++) {
        for (int j = 0; j < spalten; j++) {
            erzeugte[i][j] = (int) (101 * Math.random());
        }
    }

    return erzeugte;
}

public static void zeigeMatrix(int[][] a) {
    for (int i = 0; i < a.length; i++) {
        for (int j = 0; j < a[0].length; j++) {
            System.out.print(a[i][j] + "\t");
        }
        System.out.println();
    }
}

```



```

    }

    public static void treffeAuswahl(String operation, int[][] originalmatrix) {
        switch (operation.toLowerCase()) {

            case "t": {
                System.out.println("Transponierte Matrix: ");
                zeigeMatrix(transponieren(originalmatrix));
                break;
            }
            case "z": {
                System.out.println("Zeilensumme: ");
                for (int item : zeilensumme(originalmatrix)) {
                    System.out.println(item);
                }
                break;
            }
            case "s": {
                System.out.println("Spaltensumme: ");
                for (int item : spaltensumme(originalmatrix)) {
                    System.out.print(item + "\t");
                }
                break;
            }
            case "p": {
                System.out.println("Spurberechnung: ");
                System.out.println("Die Spur ist: " + spur(originalmatrix));
                break;
            }
            default:
                System.out.println("Fehlerhafte Eingabe: " + operation);
                System.out.println("Welche Operation wollen Sie durchfhren?");
                System.out.println("T:  Transponieren");
                System.out.println("Z:  Zeilensumme");
                System.out.println("S:  Spaltensumme");
                System.out.println("P:  Spurberechnung");
                operation=Input.readString("Whllen Sie eine der 4 Operationen: ");
                treffeAuswahl(operation,originalmatrix);

        }

    }

}

}

```

Package: OOP

Klasse: Main

```

package OOP;

import javax.swing.SwingUtilities;

public class Main {

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            MatrixView view = new MatrixView();
            new MatrixController(view);
            view.setVisible(true);
        });
    }

}

```

Klasse: Matrix

```

package OOP;

import java.util.Arrays;

public class Matrix {

    private double[][] data;

    public Matrix(int zeilen, int spalten) {
        if (zeilen <= 0 || spalten <= 0) {
            throw new MatrixParseException("Ungltige Matrixgre");
        }
        data = new double[zeilen][spalten];
    }

    public Matrix(double[][] data) {

        if (data == null || data.length == 0 || data[0].length == 0) {
            throw new MatrixParseException("Matrix darf nicht leer sein.");
        }

        int cols = data[0].length;

        for (int i = 1; i < data.length; i++) {
            if (data[i].length != cols) {
                throw new MatrixParseException("Ungltige Matrixform.");
            }
        }
    }
}

```

```

        this.data = new double[data.length][cols];

        for (int i = 0; i < data.length; i++) {
            System.arraycopy(data[i], 0, this.data[i], 0, cols);
        }
    }

    public int zeilen() {
        return data.length;
    }

    public int spalten() {
        return data[0].length;
    }

    public double get(int i, int j) {
        return data[i][j];
    }

    public void set(int i, int j, double value) {
        data[i][j] = value;
    }

    public Matrix add(Matrix other) {

        if (zeilen() != other.zeilen() || spalten() != other.spalten()) {
            throw new MatrixDimensionException("Dimension mismatch");
        }

        Matrix result = new Matrix(zeilen(), spalten());

        for (int i = 0; i < zeilen(); i++) {
            for (int j = 0; j < spalten(); j++) {
                result.set(i, j, this.get(i, j) + other.get(i, j));
            }
        }

        return result;
    }

    public Matrix subtract(Matrix other) {

        if (zeilen() != other.zeilen() || spalten() != other.spalten()) {
            throw new MatrixDimensionException("Dimension mismatch");
        }

        Matrix result = new Matrix(zeilen(), spalten());

        for (int i = 0; i < zeilen(); i++) {
            for (int j = 0; j < spalten(); j++) {
                result.set(i, j, this.get(i, j) - other.get(i, j));
            }
        }
    }

```

```

        return result;
    }

    public Matrix multiply(Matrix other) {

        if (this.spalten() != other.zeilen()) {
            throw new MatrixDimensionException("Dimension mismatch");
        }

        Matrix result = new Matrix(this.zeilen(), other.spalten());

        for (int i = 0; i < this.zeilen(); i++) {
            for (int j = 0; j < other.spalten(); j++) {

                double sum = 0;

                for (int k = 0; k < this.spalten(); k++) {
                    sum += this.get(i, k) * other.get(k, j);
                }

                result.set(i, j, sum);
            }
        }

        return result;
    }

    public double trace() {
        if (zeilen() != spalten()) {
            throw new MatrixDimensionException("Matrix muss quadratisch sein");
        }
        double spur = 0;
        for (int i = 0; i < zeilen(); i++) {
            spur += this.get(i, i);
        }
        return spur;
    }

    public Matrix transpose() {
        Matrix transponierte = new Matrix(spalten(), zeilen());
        for (int i = 0; i < spalten(); i++) {
            for (int j = 0; j < zeilen(); j++) {
                transponierte.set(i, j, this.get(j, i));
            }
        }
        return transponierte;
    }

    public Matrix scalarMultiply(double scalar) {
        Matrix result = new Matrix(zeilen(), spalten());
        for (int i = 0; i < zeilen(); i++) {
            for (int j = 0; j < spalten(); j++) {
                result.set(i, j, scalar * this.get(i, j));
            }
        }
    }

```

```

    }
}
return result;
}

public double determinant() {
    if (this.zeilen() != this.spalten()) {
        throw new MatrixDimensionException("Die Determinante ist nur fuer
quadratische Matrizen definiert.");
    }

    if (this.zeilen() == 1) {
        return this.get(0, 0);
    }

    if (this.zeilen() == 2) {
        return this.get(0, 0) * this.get(1, 1) - this.get(0, 1) * this.get(1,
0);
    }

    double summe = 0.0;
    for (int j = 0; j < this.spalten(); j++) {
        double vorzeichen = sign(0, j);
        summe += vorzeichen * this.get(0, j) * subMatrix(0, j).determinant();
    }
    return summe;
}

public Matrix inverse() {

    if (zeilen() != spalten()) {
        throw new MatrixDimensionException("Die Inverse ist nur fuer
quadratische Matrizen definiert.");
    }

    double det = determinant();

    if (det == 0) {
        throw new MatrixNotInvertibleException("Die Matrix ist nicht
invertierbar, da die Determinante 0 ist.");
    }

    if (zeilen() == 1) {
        Matrix result = new Matrix(1, 1);
        result.set(0, 0, 1.0 / get(0, 0));
        return result;
    }

    return adjugate().scalarMultiply(1.0 / det);
}

private Matrix subMatrix(int zeile, int spalte) {
    Matrix result = new Matrix(this.zeilen() - 1, this.spalten() - 1);

```

```

        int neueZeile = 0;
        for (int i = 0; i < this.zeilen(); i++) {
            if (i == zeile) {
                continue;
            }

            int neueSpalte = 0;
            for (int j = 0; j < this.spalten(); j++) {
                if (j == spalte) {
                    continue;
                }

                result.set(neueZeile, neueSpalte, this.get(i, j));
                neueSpalte++;
            }
            neueZeile++;
        }

        return result;
    }

    private double sign(int row, int col) {
        return ((row + col) % 2 == 0) ? 1.0 : -1.0;
    }

    private Matrix adjugate() {
        int n = this.zeilen();
        Matrix kofaktoren = new Matrix(n, n);

        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                Matrix minor = subMatrix(i, j);
                double vorzeichen = sign(i, j);
                kofaktoren.set(i, j, vorzeichen * minor.determinant());
            }
        }

        return kofaktoren.transpose();
    }

    @Override
    public String toString() {
        StringBuilder sb = new StringBuilder();

        for (int i = 0; i < zeilen(); i++) {
            for (int j = 0; j < spalten(); j++) {
                sb.append(String.format("%10.2f", this.get(i, j)));
            }
            sb.append("\n");
        }
        return sb.toString();
    }
}

```

```

@Override
public int hashCode() {
    final int prime = 31;
    int result = 1;
    result = prime * result + Arrays.deepHashCode(data);
    return result;
}

@Override
public boolean equals(Object obj) {
    if (this == obj)
        return true;
    if (obj == null)
        return false;
    if (getClass() != obj.getClass())
        return false;
    Matrix other = (Matrix) obj;
    return Arrays.deepEquals(data, other.data);
}
}

```

Klasse: MatrixController

```

package OOP;

public class MatrixController {

    private final MatrixView view;

    public MatrixController(MatrixView view) {
        this.view = view;

        this.view.setAddActionListener(e -> handleAdd());
        this.view.setSubtractActionListener(e -> handleSubtract());
        this.view.setMultiplyActionListener(e -> handleMultiply());
        this.view.setInverseActionListener(e -> handleInverse());
        this.view.setTransposeActionListener(e -> handleTranspose());
        this.view.setDeterminantActionListener(e -> handleDeterminant());
        this.view.setTraceActionListener(e -> handleTrace());
    }
}

```

```
private void handleAdd() {
    try {
        Matrix a = view.getMatrixA();
        Matrix b = view.getMatrixB();
        Matrix result = a.add(b);
        view.showResult(result.toString());
    } catch (MatrixException ex) {
        view.showError(ex.getMessage());
    }
}

private void handleSubtract() {
    try {
        Matrix a = view.getMatrixA();
        Matrix b = view.getMatrixB();
        Matrix result = a.subtract(b);
        view.showResult(result.toString());
    }
    catch (MatrixException ex) {
        view.showError(ex.getMessage());
    }
}

private void handleMultiply() {
    try {
        Matrix a = view.getMatrixA();
        Matrix b = view.getMatrixB();
        Matrix result = a.multiply(b);
        view.showResult(result.toString());
    } catch (MatrixException ex) {
        view.showError(ex.getMessage());
    }
}

private void handleInverse() {
    try {
        Matrix a = view.getMatrixA();
        Matrix result = a.inverse();
        view.showResult(result.toString());
    } catch (MatrixException ex) {
        view.showError(ex.getMessage());
    }
}

private void handleTranspose() {
    try {
        Matrix a = view.getMatrixA();
        Matrix result = a.transpose();
        view.showResult(result.toString());
    }
}
```



```

        } catch (MatrixException ex) {
            view.showError(ex.getMessage());
        }
    }

    private void handleTrace() {
        try {
            Matrix a = view.getMatrixA();
            double result = a.trace();
            view.showResult(String.valueOf(result));
        } catch (MatrixException ex) {
            view.showError(ex.getMessage());
        }
    }

    private void handleDeterminant() {
        try {
            Matrix a = view.getMatrixA();
            double result = a.determinant();
            view.showResult(String.valueOf(result));

        } catch (MatrixException ex) {
            view.showError(ex.getMessage());
        }
    }
}

```

Klasse: MatrixDimensionException

```

package OOP;

public class MatrixDimensionException extends MatrixException {
    public MatrixDimensionException(String message) {
        super(message);
    }
}

package OOP;

public class MatrixException extends RuntimeException {

    public MatrixException(String nachricht) {
        super(nachricht);
    }
}

```

Klasse: MatrixNotInvertibleException

```

package OOP;

```

```

public class MatrixNotInvertibleException extends MatrixException {
    public MatrixNotInvertibleException(String message) {
        super(message);
    }
}

```

Klasse: MatrixParseException

```

package OOP;

public class MatrixParseException extends MatrixException {
    public MatrixParseException(String message) {
        super(message);
    }
}

```

Klasse: MatrixView

```

package OOP;

import java.awt.BorderLayout;
import java.awt.GridLayout;
import java.awt.event.ActionListener;

import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JOptionPane;
import javax.swing.JPanel;
import javax.swing.JScrollPane;
import javax.swing.JTextArea;

public class MatrixView extends JFrame {

    private final JTextArea matrixATextArea;
    private final JTextArea matrixBTextArea;
    private final JTextArea resultTextArea;

    private final JButton addButton;
    private final JButton subtractButton;
    private final JButton multiplyButton;
    private final JButton transposeButton;
    private final JButton traceButton;
    private final JButton determinantButton;
    private final JButton inverseButton;

    public MatrixView() {
        setTitle("Matrizenrechner");
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }
}

```

```

setSize(900, 600);
setLocationRelativeTo(null);

matrixATextArea = new JTextArea(8, 20);
matrixBTextArea = new JTextArea(8, 20);
resultTextArea = new JTextArea(10, 30);
resultTextArea.setEditable(false);

addButton = new JButton("+");
subtractButton = new JButton("-");
multiplyButton = new JButton("*");
transposeButton = new JButton("T");
traceButton = new JButton("Spur");
determinantButton = new JButton("Determinante");
inverseButton = new JButton("Inverse");

JPanel inputPanel = new JPanel(new GridLayout(1, 2, 10, 10));
inputPanel.add(new JScrollPane(matrixATextArea));
inputPanel.add(new JScrollPane(matrixBTextArea));

JPanel buttonPanel = new JPanel(new GridLayout(2, 4, 10, 10));
buttonPanel.add(addButton);
buttonPanel.add(subtractButton);
buttonPanel.add(multiplyButton);
buttonPanel.add(transposeButton);
buttonPanel.add(traceButton);
buttonPanel.add(determinantButton);
buttonPanel.add(inverseButton);

JPanel resultPanel = new JPanel(new BorderLayout());
resultPanel.add(new JLabel("Ergebnis:"), BorderLayout.NORTH);
resultPanel.add(new JScrollPane(resultTextArea), BorderLayout.CENTER);

setLayout(new BorderLayout(10, 10));
add(inputPanel, BorderLayout.NORTH);
add(buttonPanel, BorderLayout.CENTER);
add(resultPanel, BorderLayout.SOUTH);
}

public Matrix getMatrixA() {
    return parseMatrix(matrixATextArea.getText());
}

public Matrix getMatrixB() {
    return parseMatrix(matrixBTextArea.getText());
}

public void showResult(String text) {
    resultTextArea.setText(text);
}

public void showError(String message) {
    JOptionPane.showMessageDialog(this, message, "Fehler",
JOptionPane.ERROR_MESSAGE);

```

```

    }

    public void setAddActionListener(ActionListener listener) {
        addButton.addActionListener(listener);
    }

    public void setSubtractActionListener(ActionListener listener) {
        subtractButton.addActionListener(listener);
    }

    public void setMultiplyActionListener(ActionListener listener) {
        multiplyButton.addActionListener(listener);
    }

    public void setTransposeActionListener(ActionListener listener) {
        transposeButton.addActionListener(listener);
    }

    public void setTraceActionListener(ActionListener listener) {
        traceButton.addActionListener(listener);
    }

    public void setDeterminantActionListener(ActionListener listener) {
        determinantButton.addActionListener(listener);
    }

    public void setInverseActionListener(ActionListener listener) {
        inverseButton.addActionListener(listener);
    }

    private Matrix parseMatrix(String input) {
        String[] rows = input.trim().split("\\n");
        int zeilen = rows.length;
        String[] firstRow = rows[0].trim().split("\\s+");
        int spalten = firstRow.length;

        double[][] werte = new double[zeilen][spalten];

        for (int i = 0; i < zeilen; i++) {
            String[] values = rows[i].trim().split("\\s+");
            for (int j = 0; j < spalten; j++) {
                werte[i][j] = Double.parseDouble(values[j].replace(",", "."));
            }
        }

        return new Matrix(werte);
    }
}

```

Package: PiShooter

Klasse: PiShooter

```

package PiShooter;

import IO.Input;

public class PiShooter {

    public static void main(String[] args) {
        double x = 0.0;
        double y = 0.0;
        int counter = 0;

        int limit;
        do {
            limit = Input.readInt("Geben Sie die Anzahl an Punkten an: ");
            if (limit < 1)
                System.out.println("Ungltige Eingabe: " + limit);
        } while (limit < 1);

        for (int i = 0; i < limit; i++) {
            x = Math.random();
            y = Math.random();
            if (x * x + y * y < 1.0) {
                counter++;
            }
        }
        double piSimmuliert = (4.0 * counter) / limit;
        System.out.println("Im Vergleich zum Litereturwert " + Math.PI + ", ");
        System.out.println("ergab unser Wert aus der Simulation: " +
piSimmuliert);
        System.out.println("Bis auf 4 Stellen genau dauerte:
"+zaehleBisPiAbweichung()+" Versuche");

    }

    public static int zaehleBisPiAbweichung() {
        int counter = 0;
        int zaehler = 1;
        double x;
        double y;
        while (true) {
            x = Math.random();
            y = Math.random();
            if (x * x + y * y < 1.0) {
                counter++;
            }
            zaehler++;
            if ((4.0 * counter) / zaehler == 3.141) {
                break;
            }
        }
        return zaehler;
    }
}

```

```
}
```

Package: Politikergehaelter

Klasse: Gesamteinkommen

```
package Politikergehaelter;

import IO.Input;

public class Gesamteinkommen {

    private static final double REDENHONORAR = 500.0;
    private static final double ZUSATZ = 200.0;
    private static final double GRUNDGEHALT = 100000.0;
    private static final double pauschaleHerrKleverle=40000.0;
    private static final double pauschaleFrauKleverle=50000.0;
    private static final double SEKRETERIATSARBEIT=200.0;

    public static void main(String[] args) {

        int anzahlRedenHerrK;
        do{
            anzahlRedenHerrK=Input.readInt("Geben Sie die Anzahl der Reden von
Herrn Kleverle ein: ");
            if(anzahlRedenHerrK<0)
                System.out.println("Fehlerhafte Eingabe: "+anzahlRedenHerrK);
        }while(anzahlRedenHerrK<0);

        int anzahlRedenFrauK;
        do{
            anzahlRedenFrauK=Input.readInt("Geben Sie die Anzahl der Reden von
Frau Kleverle ein: ");
            if(anzahlRedenFrauK<0)
                System.out.println("Fehlerhafte Eingabe: "+anzahlRedenFrauK);
        }while (anzahlRedenFrauK<0);

        int anzahlSekreteriatstundenHerrK;
        do{
            anzahlSekreteriatstundenHerrK=Input.readInt("Geben Sie die Anzahl der
Stunden Sekretariatsarbeit von Herrn Kleverle ein: ");
            if(anzahlSekreteriatstundenHerrK<0)
                System.out.println("Fehlerhafte Eingabe:
"+anzahlSekreteriatstundenHerrK);
        }while (anzahlSekreteriatstundenHerrK<0);

        int anzahlSekreteriatstundenFrauK;
        do{
            anzahlSekreteriatstundenFrauK=Input.readInt("Geben Sie die Anzahl der
Stunden Sekretariatsarbeit von Frau Kleverle ein: ");
```

```

        if (anzahlSekreteriatstundenFrauK<0)
            System.out.println("Fehlerhafte Eingab:
"+anzahlSekreteriatstundenFrauK);
        }while (anzahlSekreteriatstundenFrauK<0);

        double gemeinsamesEinkommen=0.0;

        gemeinsamesEinkommen= pauschaleFrauKleverle +pauschaleHerrKleverle +
2*GRUNDGEHALT+ berechneRedeneinkommen(anzahlRedenHerrK)

+berechneRedeneinkommen(anzahlRedenFrauK)+berechneSekreteriatseinkommen(anzahlSkr
eteriatstundenHerrK)

+berechneSekreteriatseinkommen(anzahlSekreteriatstundenFrauK);

        System.out.printf("Das gemeinsame Einkommen betrgt: %.2f
\n",gemeinsamesEinkommen);

    }

    public static double berechneRedeneinkommen(int anzahl) {
        double redenEinkommen=0.0;
        if (anzahl<0) {
            System.out.println("Fehlerhafte Eingabe!");
        }
        else if (anzahl <=5) {
            redenEinkommen=REDENHONORAR*anzahl;
        }
        else {
            redenEinkommen=5*REDENHONORAR+(REDENHONORAR+ZUSATZ)*(anzahl-5);
        }
        return redenEinkommen;
    }

    public static double berechneSekreteriatseinkommen(int stunden) {
        return SEKRETERIATSARBEIT*stunden;
    }

}

```

Package: RandomChecker

Klasse: RandomChecker

```

package RandomChecker;

import IO.Input;

```

```

public class RandomChecker {

    public static void main(String[] args) {

        int anzahl;
        do {
            anzahl = Input.readInt("Geben Sie die Anzahl der Zufallszahlen ein:
");
            if (anzahl < 1) {
                System.out.println("Fehlerhafte Eingabe: " + anzahl);
            }
        } while (anzahl < 1);

        int zuZaehlendeZahl;
        do {
            zuZaehlendeZahl = Input.readInt(
                "Geben Sie die zu zhlende Zahl zwischen 0 und 9 (inclusive)
(fr alle geben Sie die -1 ein) an: ");
            if (zuZaehlendeZahl < -1 || zuZaehlendeZahl > 9)
                System.out.println("Fehlerhafte Eingabe: " + zuZaehlendeZahl);
        } while (zuZaehlendeZahl < -1 || zuZaehlendeZahl > 9);

        System.out.println("Betrachtet werden " + anzahl + " Zahlen zwischen 0 und
9: ");

        int[] array = createRandom(anzahl);
        int begin = zuZaehlendeZahl;
        int ende = zuZaehlendeZahl;
        if (zuZaehlendeZahl == -1) {
            begin = 0;
            ende = 9;
        }
        for (int i = begin; i <= ende; i++) {
            int absolut = check(array, i);
            double durchschnitt = average(array);
            double relativ = 100.0 * absolut / anzahl;
            System.out.printf(
                "Die Zahl %td kam %td mal vor, das entspricht %t4.2f
Prozent, der Durschnitt der Zahlen betrgt: %t%.2f\n", i,
                absolut, relativ, durchschnitt);
        }
    }

    public static int[] createRandom(int n) {
        int[] arr = new int[n];
        for (int i = 0; i < n; i++) {
            arr[i] = (int) (10 * Math.random() / 1);
        }
        return arr;
    }

    public static int check(int[] r, int z) {
        int counter = 0;

```



```

        for (int item : r) {
            if (item == z) {
                counter++;
            }
        }
        return counter;
    }

    public static double average(int[] r) {
        double summe = 0;
        for (int item : r) {
            summe += item;
        }
        return (summe / r.length);
    }
}

```

Package: Studenten_haupt_verwaltung

Klasse: Lehrveranstaltung

```

package Studenten_haupt_verwaltung;

public class Lehrveranstaltung {

    private int id;
    private String bezeichnung;
    private Pruefung pruef;
    public Lehrveranstaltung(int id, String bezeichnung) {
        this.id = id;
        this.bezeichnung = bezeichnung;
    }
    public int getId() {
        return id;
    }
    public String getBezeichnung() {
        return bezeichnung;
    }
    @Override
    public String toString() {
        return "Lehrveranstaltung [id=" + id + ", bezeichnung=" + bezeichnung + ",
pruef=" + pruef + " ]";
    }

}

```

Klasse: Pruefung

```
package Studenten_haupt_verwaltung;

public class Pruefung {

    private int semester;
    private float note;
    private boolean bestanden;
    private Lehrveranstaltung lv;

    public Pruefung(Lehrveranstaltung lv) {
        this.lv = lv;
    }

    public boolean teilnehmen(float note) {
        this.note=note;
        if(note<=4.0) {
            return true;
        }
        return false;
    }

    public Lehrveranstaltung getLv() {
        return lv;
    }

    public void setLv(Lehrveranstaltung lv) {
        this.lv = lv;
    }

    public int getSemester() {
        return semester;
    }

    public float getNote() {
        return note;
    }
}
```

```

    }

    public boolean isBestanden() {
        return bestanden;
    }

    @Override
    public String toString() {
        return "Pruefung [semester=" + semester + ", note=" + note + ",
bestanden=" + bestanden + ", lv=" + lv + "]\n";
    }

}

```

Klasse: Student

```

package Studenten_haupt_verwaltung;

import java.util.Arrays;

public class Student {

    private int id;
    private String name;
    private String studiengang;
    private Pruefung[] pruefungen=new Pruefung[100];
    // for ( int i=0;i< pruefungen.length;i++) {
    //     pruefungen[i]= new Pruefung();
    // }
    // private static int counter=0;

    public Student(int id) {
        this.id = id;
        this.pruefungen = new Pruefung[100];
    }

    public int getId() {
        return id;
    }
}

```

```

    }
    public void setId(int id) {
        if (isValidId(id)){
            this.id = id;
        }
        else {
            System.out.println("Keine gltge Id!" +id+"\nId muss zwischen 10000 und
99999 sein");
        }
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }

    private boolean isValidId(int id) {
        return (id>=10000 && id<=99999);
    }

    public void datenAusgeben() {
        System.out.println("Student [Matrikelnummer: " + id + ", Name: " + name +
" ]");
    }

    public boolean lvBelegen(Lehrveranstaltung lv) {
        for (Pruefung p : pruefungen) {
            if (p != null && p.getLv() == lv) {
                return false;
            }
        }

        for (int i = 0; i < pruefungen.length; i++) {
            if (pruefungen[i] == null) {
                pruefungen[i] = new Pruefung(lv);
                return true;
            }
        }

        return false;
    }

    public boolean pruefungTeilnehmen(Lehrveranstaltung lv, float note) {
        for (Pruefung p : pruefungen) {
            if (p != null && p.getLv() == lv) {
                p.teilnehmen(note);
                return true;
            }
        }
        return false;
    }

    public float notendurchschnitt(int semester) {

```

```

        float durchschnitt=1.0f;
        float summe=0.0f;
        int anzahl=0;
        for (Pruefung p : pruefungen) {
            if(p.getSemester()==semester) {
                summe+=p.getNote();
                anzahl++;
            }
        }
        durchschnitt=summe/anzahl;

        return durchschnitt;
    }

    public float notendurchschnitt() {
        float durchschnitt=1.0f;
        float summe=0.0f;
        int anzahl=0;
        for (Pruefung p:pruefungen) {
            if(p !=null && p.getNote() > 0) {
                summe+=p.getNote();
                anzahl++;
            }
        }
        durchschnitt=summe/anzahl;

        return durchschnitt;
    }

    @Override
    public String toString() {
        return "Student [id=" + id + ", name=" + name + ", studiengang=" +
        studiengang + ", pruefungen="
            + Arrays.toString(pruefungen) + "];"
    }

}

```

Klasse: StudentenVerwaltung

```

package Studenten_haupt_verwaltung;

public class StudentenVerwaltung {

    public static void main(String[] args) {
        Student[] studenten= new Student[5] ;
    }
}

```

```

Lehrveranstaltung lv1=new Lehrveranstaltung(101, "Programmieren mit
Java");
Lehrveranstaltung lv2= new Lehrveranstaltung(102, "UML");
Lehrveranstaltung lv3 = new Lehrveranstaltung(103, "Theoretische Physik
1");

Pruefung p1= new Pruefung(lv1);
Pruefung p2 = new Pruefung(lv2);
Pruefung p3 = new Pruefung(lv3);

Pruefung[] pruef = new Pruefung[100];
pruef[0]=p1;
pruef[1]=p2;
pruef[2]=p3;

for (int i=0;i<5;i++) {
    studenten[i]=new Student(10000+i);
}

//
// studenten[0].setId(10000);
// studenten[1].setId(10001);
// studenten[2].setId(10002);
// studenten[3].setId(10003);
// studenten[4].setId(10004);
//
studenten[0].setName("Hans Wurst");
studenten[1].setName("Gerd Mller");
studenten[2].setName("Jogi Lw");
studenten[3].setName("Julian Nagelsmann");
studenten[4].setName("Berti Vogts");

//
// for(Student st:studenten) {
//     st.datenAusgeben();
// }

// studenten[0] = new Student(10000,null);

for (Student st : studenten) {
    st.lvBelegen(lv3);
    st.lvBelegen(lv1);
    st.lvBelegen(lv2);
}

studenten[0].pruefungTeilnehmen(lv3, 5.1f);
studenten[0].pruefungTeilnehmen(lv2, 2.4f);
studenten[0].pruefungTeilnehmen(lv1, 3.3f);

studenten[1].pruefungTeilnehmen(lv3, 1.6f);
studenten[1].pruefungTeilnehmen(lv2, 3.1f);
studenten[1].pruefungTeilnehmen(lv1, 4.2f);

```

```

        studenten[2].pruefungTeilnehmen(lv3, 3.6f);
        studenten[2].pruefungTeilnehmen(lv2, 2.2f);
        studenten[2].pruefungTeilnehmen(lv1, 3.4f);

        studenten[3].pruefungTeilnehmen(lv1, 2.1f);
        studenten[3].pruefungTeilnehmen(lv2, 2.3f);
        studenten[3].pruefungTeilnehmen(lv3, 2.5f);

        studenten[4].pruefungTeilnehmen(lv3, 1.2f);
        studenten[4].pruefungTeilnehmen(lv1, 2.1f);
        studenten[4].pruefungTeilnehmen(lv2, 1.7f);

        System.out.println("notendurchschnitt:
"+studenten[0].notendurchschnitt());

        for (Student st: studenten) {
            System.out.println(st.toString());
        }

    }

}

```

Package: Studenten_haupt_verwaltung_mit_Collections

Klasse: CollecionTest

```

package Studenten_haupt_verwaltung_mit_Collections;

import java.util.Collections;
import java.util.Comparator;
import java.util.LinkedList;

public class CollecionTest {

    public static void main(String[] args) {

        LinkedList<Student> studenten = new LinkedList<>();

        for (int i = 10000; i < 10005; i++) {
            studenten.add(new Student(i));
        }

        studenten.add(new Student(50000));
        studenten.add(new Student(15000));
    }
}

```

```

studenten.add(new Student(11000));

studenten.get(0).setName("Hans Wurst");
studenten.get(1).setName("Gerd Mller");
studenten.get(2).setName("Jogi Lw");
studenten.get(3).setName("Julian Nagelsmann");
studenten.get(4).setName("Berti Vogts");
studenten.get(5).setName("Nachm Sortieren Letzter");
studenten.get(6).setName("Nachm und Vor dem Sortieren Vorletzter");
studenten.get(7).setName("Vor dem Sortieren Letzter");

System.out.println("====mit foreach====");
for (Student st : studenten) {
    System.out.println("Name: " + st.getName());
}

System.out.println();
System.out.println("====mit normaler for-Schleife====");
for (int i = 0; i < studenten.size(); i++) {
    System.out.println("Name: " + studenten.get(i).getName());
}
System.out.println("=".repeat(65));

Comparator<Student> myComparator = new IdVergleicher();
Collections.sort(studenten, myComparator);
System.out.println(studenten.toString());
for (int i = 0; i < studenten.size(); i++) {
    System.out.println("Name: " + studenten.get(i).getName());
}

System.out.println("=".repeat(65));
System.out.println("Sortiert nach Namen: ");
Comparator<Student> myComparator2 = new NameVergleicher();
Collections.sort(studenten, myComparator2);
System.out.println(studenten.toString());
for (int i = 0; i < studenten.size(); i++) {
    System.out.println("Name: " + studenten.get(i).getName());
}

}

}

```

Klasse: CollectionTest2

```

package Studenten_haupt_verwaltung_mit_Collections;

import java.util.HashMap;

```



```

import java.util.Map;

public class CollectionTest2 {

    public static void main(String[] args) {
        Map<Student,Integer> studenten= new HashMap<>();
        // for (int i=10000;i<10005;i++) {
        //     studenten.put(new Student(i),i);
        // }
        //
        // studenten.get(0).setName("Hans Wurst");
        // studenten.get(1).setName("Gerd Mller");
        // studenten.get(2).setName("Jogi Lw");
        // studenten.get(3).setName("Julian Nagelsmann");
        // studenten.get(4).setName("Berti Vogts");

        Student st1= new Student(10000);
        st1.setName("HansWurst");
        Student st2 = new Student(10001);
        st2.setName("Gerd Mller");
        Student st3= new Student(10002);
        st3.setName("Jogi Lw");
        Student st4= new Student(10003);
        st4.setName("Julian Nagelsmann");
        Student st5 = new Student(10004);
        st5.setName("Berti Vogts");

        studenten.put(st1,10000);
        studenten.put(st2,10001);
        studenten.put(st3,10002);
        studenten.put(st4,10003);
        studenten.put(st5,10004);

        studenten.forEach((k,v)-> System.out.println(k+": Matrikelnummer ="
+ v));

    }

}

```

Klasse: Lehrveranstaltung

```
package Studenten_haupt_verwaltung_mit_Collections;
```

```

public class Lehrveranstaltung {

    private int id;
    private String bezeichnung;
    private Pruefung pruef;
    public Lehrveranstaltung(int id, String bezeichnung) {
        this.id = id;
        this.bezeichnung = bezeichnung;
    }
    public int getId() {
        return id;
    }
    public String getBezeichnung() {
        return bezeichnung;
    }
    @Override
    public String toString() {
        return "Lehrveranstaltung [id=" + id + ", bezeichnung=" + bezeichnung + ",
pruef=" + pruef + "]";
    }

}

```

Klasse: Student

```

package Studenten_haupt_verwaltung_mit_Collections;

import java.util.Arrays;

public class Student {

    private int id;
    private String name;
    private String studiengang;
    private Pruefung[] pruefungen=new Pruefung[3];
    // for ( int i=0;i< pruefungen.length;i++) {
    //     pruefungen[i]= new Pruefung();
    // }
    // private static int counter=0;

    public Student(int id) {
        this.id = id;
        this.pruefungen = new Pruefung[3];
    }

    public int getId() {
        return id;
    }
    public void setId(int id) {

```

```

        if (isValidId(id)){
            this.id = id;
        }
        else {
            System.out.println("Keine gltge Id!" +id+"\nId muss zwischen 10000 und
99999 sein");
        }
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }

    private boolean isValidId(int id) {
        return (id>=10000 && id<=99999);
    }

    public void datenAusgeben() {
        System.out.println("Student [Matrikelnummer: " + id + ", Name: " + name +
" ]");
    }

    public boolean lvBelegen(Lehrveranstaltung lv) {
        for (Pruefung p : pruefungen) {
            if (p != null && p.getLv() == lv) {
                return false;
            }
        }

        for (int i = 0; i < pruefungen.length; i++) {
            if (pruefungen[i] == null) {
                pruefungen[i] = new Pruefung(lv);
                return true;
            }
        }

        return false;
    }

    public boolean pruefungTeilnehmen(Lehrveranstaltung lv, float note) {
        for (Pruefung p : pruefungen) {
            if (p != null && p.getLv() == lv) {
                p.teilnehmen(note);
                return true;
            }
        }
        return false;
    }

    public float notendurchschnitt(int semester) {
        float durchschnitt=1.0f;
        float summe=0.0f;
    }

```

```

        int anzahl=0;
        for (Pruefung p : pruefungen) {
            if(p.getSemester()==semester) {
                summe+=p.getNote();
                anzahl++;
            }
        }
        durchschnitt=summe/anzahl;

        return durchschnitt;
    }

    public float notendurchschnitt() {
        float durchschnitt=1.0f;
        float summe=0.0f;
        int anzahl=0;
        for (Pruefung p:pruefungen) {
            if(p !=null && p.getNote() > 0) {
                summe+=p.getNote();
                anzahl++;
            }
        }
        durchschnitt=summe/anzahl;

        return durchschnitt;
    }

    @Override
    public String toString() {
        return "Student [id=" + id + ", name=" + name + ", studiengang=" +
studiengang + ", pruefungen="
            + Arrays.toString(pruefungen) + "]";
    }

}

```

Klasse: StudentenVerwaltung

```

package Studenten_haupt_verwaltung_mit_Collections;

public class StudentenVerwaltung {

    public static void main(String[] args) {
        Student[] studenten= new Student[5] ;

        Lehrveranstaltung lv1=new Lehrveranstaltung(101, "Programmieren mit
Java");
    }
}

```

```

Lehrveranstaltung lv2= new Lehrveranstaltung(102, "UML");
Lehrveranstaltung lv3 = new Lehrveranstaltung(103, "Theoretische Physik
1");

Pruefung p1= new Pruefung(lv1);
Pruefung p2 = new Pruefung(lv2);
Pruefung p3 = new Pruefung(lv3);

Pruefung[] pruef = new Pruefung[100];
pruef[0]=p1;
pruef[1]=p2;
pruef[2]=p3;

for (int i=0;i<5;i++) {
    studenten[i]=new Student(10000+i);
}

//
// studenten[0].setId(10000);
// studenten[1].setId(10001);
// studenten[2].setId(10002);
// studenten[3].setId(10003);
// studenten[4].setId(10004);
//
studenten[0].setName("Hans Wurst");
studenten[1].setName("Gerd Mller");
studenten[2].setName("Jogi Lw");
studenten[3].setName("Julian Nagelsmann");
studenten[4].setName("Berti Vogts");

//
// for(Student st:studenten) {
//     st.datenAusgeben();
// }

// studenten[0] = new Student(10000,null);

for (Student st : studenten) {
    st.lvBelegen(lv3);
    st.lvBelegen(lv1);
    st.lvBelegen(lv2);
}

studenten[0].pruefungTeilnehmen(lv3, 5.1f);
studenten[0].pruefungTeilnehmen(lv2, 2.4f);
studenten[0].pruefungTeilnehmen(lv1, 3.3f);

studenten[1].pruefungTeilnehmen(lv3, 1.6f);
studenten[1].pruefungTeilnehmen(lv2, 3.1f);
studenten[1].pruefungTeilnehmen(lv1, 4.2f);

studenten[2].pruefungTeilnehmen(lv3, 3.6f);

```

```

        studenten[2].pruefungTeilnehmen(lv2, 2.2f);
        studenten[2].pruefungTeilnehmen(lv1, 3.4f);

        studenten[3].pruefungTeilnehmen(lv1, 2.1f);
        studenten[3].pruefungTeilnehmen(lv2, 2.3f);
        studenten[3].pruefungTeilnehmen(lv3, 2.5f);

        studenten[4].pruefungTeilnehmen(lv3, 1.2f);
        studenten[4].pruefungTeilnehmen(lv1, 2.1f);
        studenten[4].pruefungTeilnehmen(lv2, 1.7f);

        System.out.println("notendurchschnitt:
"+studenten[0].notendurchschnitt());

        for (Student st: studenten) {
            System.out.println(st.toString());

        }

    }

}

```

Package: Taschenrechner_GUI

Klasse: Einfacher_Taschenrechner

```

package Taschenrechner_GUI;

import java.awt.EventQueue;

import javax.swing.JFrame;
import javax.swing.JPanel;
import javax.swing.border.EmptyBorder;

import javax.swing.JLabel;
import javax.swing.JButton;
import javax.swing.JTextField;
import java.awt.Font;
import java.awt.event.ActionListener;
import java.awt.event.ActionEvent;

public class Einfacher_Taschenrechner extends JFrame {

    private static final long serialVersionUID = 1L;

```

```

private JPanel contentPane;
private final JLabel lblEinfacherTaschenrechner = new JLabel("Einfacher
Taschenrechner");
private JTextField txtField1;
private JTextField txtField2;
private JTextField txtFieldErgebnis;

/**
 * Launch the application.
 */
public static void main(String[] args) {
    EventQueue.invokeLater(new Runnable() {
        public void run() {
            try {
                Einfacher_Taschenrechner frame = new
Einfacher_Taschenrechner();
                frame.setVisible(true);
            } catch (Exception e) {
                e.printStackTrace();
            }
        }
    });
}

/**
 * Create the frame.
 */
public Einfacher_Taschenrechner() {

    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setBounds(100, 100, 450, 300);
    contentPane = new JPanel();
    contentPane.setBorder(new EmptyBorder(5, 5, 5, 5));
    setContentPane(contentPane);
    contentPane.setLayout(null);

    JButton btnPlus = new JButton("+");
    btnPlus.addActionListener(new ActionListener() {
        public void actionPerformed(ActionEvent e) {

            try {
                double zahl1 = Double.parseDouble(txtField1.getText());
                double zahl2 = Double.parseDouble(txtField2.getText());

                double ergebnis = zahl1 + zahl2;

                txtFieldErgebnis.setText(String.valueOf(ergebnis));

            } catch (NumberFormatException ex) {
                txtFieldErgebnis.setText("Fehler");
            }
        }
    });
}

```

```

});
btnPlus.setBounds(291, 157, 89, 23);
contentPane.add(btnPlus);

JButton btnMinus = new JButton("-");
btnMinus.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        try {
            double zahl1 = Double.parseDouble(txtField1.getText());
            double zahl2 = Double.parseDouble(txtField2.getText());

            double ergebnis = zahl1 - zahl2;

            txtFieldErgebnis.setText(String.valueOf(ergebnis));

        } catch (NumberFormatException ex) {
            txtFieldErgebnis.setText("Fehler");
        }
    }
});
btnMinus.setBounds(291, 184, 89, 23);
contentPane.add(btnMinus);

JButton btnMal = new JButton("x");
btnMal.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        try {
            double zahl1 = Double.parseDouble(txtField1.getText());
            double zahl2 = Double.parseDouble(txtField2.getText());

            double ergebnis = zahl1 * zahl2;

            txtFieldErgebnis.setText(String.valueOf(ergebnis));

        } catch (NumberFormatException ex) {
            txtFieldErgebnis.setText("Fehler");
        }
    }
});
btnMal.setBounds(291, 210, 89, 23);
contentPane.add(btnMal);

JButton btnGeteilt = new JButton("/");
btnGeteilt.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        try {
            double zahl1 = Double.parseDouble(txtField1.getText());
            double zahl2 = Double.parseDouble(txtField2.getText());

```



```

        if (zahl2 == 0) {
            txtFieldErgebnis.setText("Division durch 0!");
            return;
        }

        double ergebnis = zahl1 / zahl2;

        txtFieldErgebnis.setText(String.valueOf(ergebnis));

    }catch (NumberFormatException ex) {
        txtFieldErgebnis.setText("Fehler");
    }

    }
});
btnGeteilt.setBounds(291, 238, 89, 23);
contentPane.add(btnGeteilt);

txtField1 = new JTextField();
txtField1.setBounds(31, 69, 86, 20);
contentPane.add(txtField1);
txtField1.setColumns(10);

txtField2 = new JTextField();
txtField2.setBounds(31, 121, 86, 20);
contentPane.add(txtField2);
txtField2.setColumns(10);

txtFieldErgebnis = new JTextField();
txtFieldErgebnis.setBounds(207, 69, 150, 55);
contentPane.add(txtFieldErgebnis);
txtFieldErgebnis.setColumns(10);

JLabel lblNewLabel = new JLabel("1. Zahl");
lblNewLabel.setBounds(31, 51, 46, 14);
contentPane.add(lblNewLabel);

JLabel lblNewLabel_1 = new JLabel("2. Zahl");
lblNewLabel_1.setBounds(31, 100, 46, 14);
contentPane.add(lblNewLabel_1);

JLabel lblNewLabel_2 = new JLabel("Ergebnis");
lblNewLabel_2.setFont(new Font("Tahoma", Font.PLAIN, 14));
lblNewLabel_2.setBounds(209, 45, 76, 23);
contentPane.add(lblNewLabel_2);
lblEinfacherTaschenrechner.setFont(new Font("Tahoma", Font.PLAIN, 14));
lblEinfacherTaschenrechner.setBounds(119, 0, 166, 32);

```

```

        contentPane.add(lblEinfacherTaschenrechner);

    }
}

```

Klasse: Einfacher_TaschenrechnerVersion2

```

package Taschenrechner_GUI;

import java.awt.EventQueue;
import javax.swing.*;
import javax.swing.border.EmptyBorder;
import java.awt.Font;

public class Einfacher_TaschenrechnerVersion2 extends JFrame {

    private JPanel contentPane;

    private JTextField txtField1;
    private JTextField txtField2;
    private JTextField txtFieldErgebnis;

    public static void main(String[] args) {
        EventQueue.invokeLater(() -> {
            try {
                Einfacher_TaschenrechnerVersion2 frame = new
Einfacher_TaschenrechnerVersion2();
                frame.setVisible(true);
            } catch (Exception e) {
                e.printStackTrace();
            }
        });
    }

    public Einfacher_TaschenrechnerVersion2() {

        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setBounds(100, 100, 450, 300);

        contentPane = new JPanel();
        contentPane.setBorder(new EmptyBorder(5, 5, 5, 5));
        contentPane.setLayout(null);
        setContentPane(contentPane);

        // Felder
        txtField1 = new JTextField();
        txtField1.setBounds(31, 69, 86, 20);
        contentPane.add(txtField1);

        txtField2 = new JTextField();
        txtField2.setBounds(31, 121, 86, 20);
    }
}

```

```

contentPane.add(txtField2);

txtFieldErgebnis = new JTextField();
txtFieldErgebnis.setBounds(207, 69, 150, 55);
contentPane.add(txtFieldErgebnis);

// Buttons
JButton btnPlus = new JButton("+");
btnPlus.setBounds(291, 157, 89, 23);
contentPane.add(btnPlus);

JButton btnMinus = new JButton("-");
btnMinus.setBounds(291, 184, 89, 23);
contentPane.add(btnMinus);

JButton btnMal = new JButton("x");
btnMal.setBounds(291, 210, 89, 23);
contentPane.add(btnMal);

JButton btnGeteilt = new JButton("/");
btnGeteilt.setBounds(291, 238, 89, 23);
contentPane.add(btnGeteilt);

// Actions
btnPlus.addActionListener(e -> berechne(new PlusStrategy()));
btnMinus.addActionListener(e -> berechne(new MinusStrategy()));
btnMal.addActionListener(e -> berechne(new MalStrategy()));
btnGeteilt.addActionListener(e -> berechne(new GeteiltStrategy()));
}

// zentrale Logik
private void berechne(OperationStrategy strategy) {

    try {
        double a = Double.parseDouble(txtField1.getText().trim());
        double b = Double.parseDouble(txtField2.getText().trim());

        if (strategy instanceof GeteiltStrategy && b == 0) {
            txtFieldErgebnis.setText("Division durch 0!");
            return;
        }

        double result = strategy.berechne(a, b);

        txtFieldErgebnis.setText(String.valueOf(result));

    } catch (NumberFormatException ex) {
        txtFieldErgebnis.setText("Fehler");
    }
}

}package LIFO_FIFO;

public class EmptyException extends Exception{

```

```

        public EmptyException(String nachricht) {
            //String message="Die Liste ist leer.Sie knnen kein Element von ener
leeren Liste lschen!";
            System.out.println(nachricht);
        }
    }
}

```

Klasse: GeteiltStrategy

```

package Taschenrechner_GUI;

public class GeteiltStrategy implements OperationStrategy {
    public double berechne(double a, double b) {
        return a / b;
    }
}package model;

public class GeteiltStrategy implements OperationStrategy {

    @Override
    public double berechne(double a, double b) {
        return a / b;
    }
}package model;

public class GeteiltStrategy implements OperationStrategy {

    @Override
    public double berechne(double a, double b) {
        return a / b;
    }
}package Studenten_haupt_verwaltung_mit_Collections;

import java.util.Comparator;

class IdVergleicher implements Comparator<Student> {

    public int compare(Student object1, Student object2) {
        Student st1=(Student) object1;
        Student st2=(Student) object2;

        if (st1.getId()<st2.getId()) {
            return -1;
        }

        if (st1.getId()>st2.getId()) {
            return 1;
        }
        return 0;
    }
}

```

```

    }

}

```

Klasse: MinusStrategy

```

package Taschenrechner_GUI;

public class MinusStrategy implements OperationStrategy {
    public double berechne(double a, double b) {
        return a - b;
    }
}package model;

public class MinusStrategy implements OperationStrategy {

    @Override
    public double berechne(double a, double b) {
        return a - b;
    }
}package model;

public class MinusStrategy implements OperationStrategy {

    @Override
    public double berechne(double a, double b) {
        return a - b;
    }
}/**
 *
 */
/**
 *
 */
module Matritzenrechner2x2 {
    requires java.desktop;
}/**
 *
 */
/**
 *
 */
module Einfacher_TaschenrechnerMVC {
    requires java.desktop;
}package Studenten_haupt_verwaltung_mit_Collections;

import java.util.Comparator;

public class NameVergleicher implements Comparator<Student> {

    @Override

```

```

        public int compare(Student o1, Student o2) {
//            if (Integer.valueOf(o2.getName())<Integer.valueOf(o2.getName())) {
//                return -1;
//            }
//            if (Integer.valueOf(o2.getName())>Integer.valueOf(o2.getName())) {
//                return 1;
//            }
//            return 0;

            return o1.getName().compareTo(o2.getName());
        }

    }
}

```

Klasse: OperationStrategy

```

package Taschenrechner_GUI;

public interface OperationStrategy {
    double berechne(double a, double b);
}

```

Klasse: PlusStrategy

```

package Taschenrechner_GUI;

public class PlusStrategy implements OperationStrategy {
    public double berechne(double a, double b) {
        return a + b;
    }
}package model;

public class PlusStrategy implements OperationStrategy {

    @Override
    public double berechne(double a, double b) {
        return a + b;
    }
}package model;

public class PlusStrategy implements OperationStrategy {

    @Override
    public double berechne(double a, double b) {
        return a + b;
    }
}package Studenten_haupt_verwaltung_mit_Collections;

```

```
public class Pruefung {

    private int semester;
    private float note;
    private boolean bestanden;
    private Lehrveranstaltung lv;

    public Pruefung(Lehrveranstaltung lv) {
        this.lv = lv;
    }

    public boolean teilnehmen(float note) {
        this.note=note;
        if(note<=4.0) {
            return true;
        }
        return false;
    }

    public Lehrveranstaltung getLv() {
        return lv;
    }

    public void setLv(Lehrveranstaltung lv) {
        this.lv = lv;
    }

    public int getSemester() {
        return semester;
    }

    public float getNote() {
        return note;
    }
}
```

```

    public boolean isBestanden() {
        return bestanden;
    }

    @Override
    public String toString() {
        return "Pruefung [semester=" + semester + ", note=" + note + ",
bestanden=" + bestanden + ", lv=" + lv + "]";
    }

}

```

Package: Timer

Klasse: CountdownWindowBuilder

```

package Timer;

import javax.swing.*;
import javax.swing.border.EmptyBorder;
import java.awt.*;
import java.awt.event.KeyEvent;

public class CountdownWindowBuilder extends JFrame {

    private JPanel contentPane;
    private JLabel titleLabel;
    private JLabel timeLabel;

    private RoundButton startPauseButton;
    private RoundButton resetButton;
    private RoundButton newButton;

    private JProgressBar progressBar;

    private Timer timer;
    private Timer pulseTimer;

    private int initialSeconds = 0;
}

```



```

private int remainingSeconds = 0;
private boolean running = false;

private JMenuItem newItem, startPauseItem, resetItem, exitItem, aboutItem;

private final Color bg = new Color(24, 24, 28);
private final Color panelBg = new Color(32, 32, 38);
private final Color text = new Color(235, 235, 235);
private final Color accent = new Color(70, 130, 255);
private final Color green = new Color(46, 125, 50);
private final Color red = new Color(198, 40, 40);

public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> {
        CountdownWindowBuilder f = new CountdownWindowBuilder();
        f.setVisible(true);
    });
}

public CountdownWindowBuilder() {

    setTitle("Countdown");
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setSize(700, 500);
    setLocationRelativeTo(null);
    setResizable(false);

    contentPane = new JPanel();
    contentPane.setBorder(new EmptyBorder(20, 20, 20, 20));
    contentPane.setBackground(bg);
    contentPane.setLayout(new BorderLayout(15, 15));
    setContentPane(contentPane);

    createMenu();

    // ===== CENTER =====
    JPanel center = new JPanel(new BorderLayout(10, 10));
    center.setOpaque(false);

    titleLabel = new JLabel("Countdown", SwingConstants.CENTER);
    titleLabel.setFont(new Font("SansSerif", Font.BOLD, 30));
    titleLabel.setForeground(text);

    timeLabel = new JLabel("00:00", SwingConstants.CENTER);
    timeLabel.setFont(new Font("SansSerif", Font.BOLD, 64));
    timeLabel.setForeground(accent);

    progressBar = new JProgressBar(0, 100);
    progressBar.setStringPainted(true);
    progressBar.setValue(100);
    progressBar.setString("100%");
    progressBar.setForeground(accent);
    progressBar.setBackground(panelBg);

```

```

center.add(titleLabel, BorderLayout.NORTH);
center.add(timeLabel, BorderLayout.CENTER);
center.add(progressBar, BorderLayout.SOUTH);

contentPane.add(center, BorderLayout.CENTER);

// ===== BUTTONS =====
JPanel buttons = new JPanel(new FlowLayout(FlowLayout.CENTER, 15, 5));
buttons.setOpaque(false);

newButton = new RoundButton("Neu");
startPauseButton = new RoundButton("Start");
resetButton = new RoundButton("Reset");

style(newButton, accent);
style(startPauseButton, green);
style(resetButton, red);

resetButton.setEnabled(false);

buttons.add(newButton);
buttons.add(startPauseButton);
buttons.add(resetButton);

contentPane.add(buttons, BorderLayout.SOUTH);

// ===== MAIN TIMER =====
timer = new Timer(1000, e -> {

    remainingSeconds--;
    updateDisplay();

    if (remainingSeconds <= 0) {
        timer.stop();
        pulseTimer.stop();

        running = false;
        startPauseButton.setText("Start");

        timeLabel.setVisible(true);
        timeLabel.setText("Fertig!");
        timeLabel.setForeground(Color.RED);

        Toolkit.getDefaultToolkit().beep();
    }
});

// ===== PULSE TIMER =====
pulseTimer = new Timer(300, e -> {

    if (remainingSeconds <= 5 && remainingSeconds > 0) {
        timeLabel.setVisible(!timeLabel.isVisible());
    } else {
        timeLabel.setVisible(true);
    }
});

```

```

    }
});

pulseTimer.start();

// ===== ACTIONS =====

newButton.addActionListener(e -> openDialog());

startPauseButton.addActionListener(e -> startPause());

resetButton.addActionListener(e -> reset());

newItem.addActionListener(e -> openDialog());
startPauseItem.addActionListener(e -> startPauseButton.doClick());
resetItem.addActionListener(e -> reset());
exitItem.addActionListener(e -> System.exit(0));

aboutItem.addActionListener(e -> JOptionPane.showMessageDialog(
    this,
    "Countdown Timer",
    "About",
    JOptionPane.INFORMATION_MESSAGE
));
}

// ===== START / PAUSE =====
private void startPause() {

    if (!running) {

        if (remainingSeconds <= 0) return;

        timer.start();
        pulseTimer.start();

        running = true;
        startPauseButton.setText("Pause");
        resetButton.setEnabled(true);

    } else {

        timer.stop();
        pulseTimer.stop();

        running = false;
        startPauseButton.setText("Fortsetzen");

        timeLabel.setVisible(true);

    }
}

// ===== RESET =====
private void reset() {

```

```

timer.stop();
pulseTimer.stop();

running = false;
remainingSeconds = initialSeconds;

updateDisplay();

timeLabel.setVisible(true);
timeLabel.setForeground(accent);

startPauseButton.setText("Start");
resetButton.setEnabled(false);
}

// ===== POPUP =====
private void openDialog() {

    JDialog d = new JDialog(this, "Neue Zeit", true);
    d.setSize(280, 160);
    d.setLocationRelativeTo(this);
    d.setLayout(new FlowLayout());

    JTextField m = new JTextField(String.valueOf(initialSeconds / 60), 3);
    JTextField s = new JTextField(String.valueOf(initialSeconds % 60), 3);

    JButton ok = new JButton("OK");

    ok.addActionListener(ev -> {
        try {

            int mm = Integer.parseInt(m.getText().trim());
            int ss = Integer.parseInt(s.getText().trim());

            if (mm < 0 || ss < 0 || ss > 59) {
                JOptionPane.showMessageDialog(d,
                    "Bitte gültige Zeit eingeben:\nMin ≥ 0\nSek 0-59",
                    "Ungültig",
                    JOptionPane.WARNING_MESSAGE);
                return;
            }

            timer.stop();
            pulseTimer.stop();

            running = false;
            startPauseButton.setText("Start");

            setTime(mm, ss);

            resetButton.setEnabled(true);

            d.dispose();

```

```

        } catch (NumberFormatException ex) {
            JOptionPane.showMessageDialog(d,
                "Bitte nur Zahlen eingeben!",
                "Fehler",
                JOptionPane.ERROR_MESSAGE);
        }
    });

    d.add(new JLabel("Min:"));
    d.add(m);
    d.add(new JLabel("Sek:"));
    d.add(s);
    d.add(ok);

    d.setVisible(true);
}

// ===== TIME =====
private void setTime(int m, int s) {

    initialSeconds = m * 60 + s;
    remainingSeconds = initialSeconds;

    updateDisplay();
}

// ===== DISPLAY =====
private void updateDisplay() {

    int min = remainingSeconds / 60;
    int sec = remainingSeconds % 60;

    timeLabel.setText(String.format("%02d:%02d", min, sec));

    // Farbe
    if (remainingSeconds <= 5 && remainingSeconds > 0) {
        timeLabel.setForeground(Color.RED);
    } else if (remainingSeconds <= 10) {
        timeLabel.setForeground(Color.RED.darker());
    } else {
        timeLabel.setForeground(accent);
    }

    if (initialSeconds > 0) {
        int percent = (int) (remainingSeconds * 100.0 / initialSeconds);
        progressBar.setValue(percent);
        progressBar.setString(percent + "%");
    }
}

// ===== STYLE =====
private void style(RoundButton b, Color c) {
    b.setBackground(c);
}

```

```

        b.setForeground(Color.WHITE);
        b.setPreferredSize(new Dimension(90, 90));
    }

    // ===== MENU =====
    private void createMenu() {

        JMenuBar bar = new JMenuBar();
        setJMenuBar(bar);

        JMenu m1 = new JMenu("Menü");
        JMenu m2 = new JMenu("About");

        bar.add(m1);
        bar.add(m2);

        newItem = new JMenuItem("Neu");
        startPauseItem = new JMenuItem("Start");
        resetItem = new JMenuItem("Reset");
        exitItem = new JMenuItem("Exit");
        aboutItem = new JMenuItem("Info");

        m1.add(newItem);
        m1.add(startPauseItem);
        m1.add(resetItem);
        m1.add(exitItem);

        m2.add(aboutItem);

        int ctrl = Toolkit.getDefaultToolkit().getMenuShortcutKeyMaskEx();

        newItem.setAccelerator(KeyStroke.getKeyStroke(KeyEvent.VK_N, ctrl));
        startPauseItem.setAccelerator(KeyStroke.getKeyStroke(KeyEvent.VK_S,
ctrl));
        resetItem.setAccelerator(KeyStroke.getKeyStroke(KeyEvent.VK_R, ctrl));
        exitItem.setAccelerator(KeyStroke.getKeyStroke(KeyEvent.VK_X, ctrl));
    }
}package Datumsberechnung;

import java.time.LocalDate;

import IO.Input;

public class Date {

    private int tag;
    private int monat;
    private int jahr;

    public Date() {
        jahr = LocalDate.now().getYear();
        monat = LocalDate.now().getMonthValue();
        tag = LocalDate.now().getDayOfMonth();
    }
}

```

```

public Date(int tag, int monat, int jahr) {

    if (check(jahr,monat,tag)) {
        this.tag = tag;
        this.monat = monat;
        this.jahr = jahr;
    }
    else {
        LocalDate d= Input.readDate("Geben Sie ein gltiges Datum ein: ");
        this.jahr=d.getYear();
        this.monat=d.getMonthValue();
        this.tag=d.getDayOfMonth();
    }
}

public boolean check(int jahr, int monat, int tag) {
    boolean isValidDate = true;
    boolean istSchaltjahr = false;
    if (jahr % 400 == 0 || jahr % 100 != 0 && jahr % 4 == 0) {
        istSchaltjahr = true;
    }
    if (jahr == 0) {
        isValidDate = false;
    }
    if (monat < 1 || monat > 12) {
        isValidDate = false;
    }
    int grenze = 0;
    switch (monat) {
        case 1, 3, 5, 7, 8, 10, 12:
            grenze = 31;
            break;
        case 4, 6, 9, 11:
            grenze = 30;
            break;
        case 2:
            grenze = istSchaltjahr ? 29 : 28;
            break;
        default:
            System.out.println("Ungltige Eingabe");
    }
    if (tag < 1 || tag > grenze) {
        isValidDate = false;
    }
    return isValidDate;
}

public int getTag() {
    return tag;
}

public int getMonat() {
    return monat;
}

```

```

    }

    public int getJahr() {
        return jahr;
    }

    @Override
    public String toString() {
        return "Date [tag=" + tag + ", monat=" + monat + ", jahr=" + jahr + "];"
    }

}

```

Package: Vorueberlegungen

Klasse: FalscheMatrixAktionException

```

package Vorueberlegungen;

public class FalscheMatrixAktionException extends Exception {

    public FalscheMatrixAktionException() {
        System.out.println("Diese Aktion ist fr Ihre Matritzen nicht erlaubt!");
    }

}

```

Klasse: KeineInverseException

```

package Vorueberlegungen;

public class KeineInverseException extends Exception {

    public KeineInverseException() {
        System.out.println("Fr diese Matrix existiert keine Inverse! Die Matrix
ist 'singular'.");
    }

}

```

Klasse: Vorueberlegungen

```

package Vorueberlegungen;

import IO.Input;

```



```

public class Vorueberlegungen {

    public static void zeigeMatrix(double[][] mat) {
        System.out.println("=====");
        for (int i = 0; i < mat.length; i++) {
            for (int j = 0; j < mat[0].length; j++) {
                System.out.printf("%8.2f", mat[i][j]);
            }
            System.out.println();
        }
        System.out.println("=====");
    }

    public static void liesMatrixEin(double[][] mat) {
        System.out.println("=====");
        for (int i = 0; i < mat.length; i++) {
            for (int j = 0; j < mat[0].length; j++) {
                mat[i][j] = Input.readDouble("Matrixelement(" + (i + 1) + " (" + (j + 1) +
"): ");
            }
        }
        System.out.println("=====");
    }

    // public static double berechneDeterminante(double[][] mat) {
    //     if(mat.length != 2 || mat[0].length != 2) {
    //         throw new UnsupportedOperationException(
    //             "Determinante aktuell nur fr 2x2");
    //     }
    //     double determinante = 0;
    //     determinante = mat[0][0]*mat[1][1] - mat[0][1]*mat[1][0];
    //     return determinante;
    // }

    public static double berechneDeterminante(double[][] mat) {
        int n = mat.length;

        // Plausibilitätsprüfung: Die Matrix muss quadratisch sein
        if (n != mat[0].length) {
            throw new IllegalArgumentException("Die Matrix muss quadratisch
sein.");
        }

        // --- BASISFALL ---
        // Wenn wir bei einer 2x2 Matrix angekommen sind, nutzen wir deine Logik
        if (n == 2) {
            return mat[0][0] * mat[1][1] - mat[0][1] * mat[1][0];
        }

        // (Optionaler Zusatz: Falls die Matrix 1x1 ist)
        if (n == 1) {

```

```

        return mat[0][0];
    }

    // --- REKURSIONSSCHRITT ---
    double determinante = 0;

    // Wir entwickeln nach der ersten Zeile (Index 0)
    for (int j = 0; j < n; j++) {
        // 1. Untermatrix erstellen (Gre (n-1) x (n-1))
        double[][] untermatrix = erzeugeUntermatrix(mat, 0, j);

        // 2. Das Vorzeichen bestimmen (Schachbrettmuster: + - + - ...)
        // Fr die erste Zeile (i=0) gilt:  $(-1)^{(0+j)}$  -> also positiv bei
geradem j, negativ bei ungeradem j
        double vorzeichen = (j % 2 == 0) ? 1.0 : -1.0;

        // 3. Rekursiver Aufruf und Aufsummieren
        determinante += vorzeichen * mat[0][j] *
berechneDeterminante(untermatrix);
    }

    return determinante;
}

// Hilfsmethode, die eine Zeile und eine Spalte aus der Matrix streicht
private static double[][] erzeugeUntermatrix(double[][] mat, int
zeileZuStreichen, int spalteZuStreichen) {
    int n = mat.length;
    double[][] untermatrix = new double[n - 1][n - 1];

    int r = 0; // Zeilen-Index fr die Untermatrix
    for (int i = 0; i < n; i++) {
        if (i == zeileZuStreichen) continue; // berspringe die Streichzeile

        int c = 0; // Spalten-Index fr die Untermatrix
        for (int j = 0; j < n; j++) {
            if (j == spalteZuStreichen) continue; // berspringe die
Streichspalte

            untermatrix[r][c] = mat[i][j];
            c++;
        }
        r++;
    }
    return untermatrix;
}

public static double berechneSpur(double[][] mat) throws
FalscheMatrixAktionException {
    if (mat.length != mat[0].length) {
        throw new FalscheMatrixAktionException();
    }
    double spur=0;
    for (int i=0;i< mat.length;i++) {

```

```

        spur+=mat[i][i];
    }
    return spur;
}

    public static double[][] berechneInverse(double[][] mat) throws
KeineInverseException{
    if(mat.length != 2 || mat[0].length != 2) {
        throw new UnsupportedOperationException(
            "Inverse aktuell nur fr 2x2");
    }
    double[][] inverse= new double[mat.length][mat[0].length];
    if (berechneDeterminante(mat)==0) {
        throw new KeineInverseException();
    }
    double det=berechneDeterminante(mat);
    inverse[0][0]= mat[1][1]/det;
    inverse[0][1]= -mat[0][1]/det;
    inverse[1][0]= -mat[1][0]/det;
    inverse[1][1]=mat[0][0]/det;
    return inverse;
}

    public static double[][] bestimmeTransponierte(double[][] mat) {
        double[][] transponierte = new double[mat[0].length][mat.length];
        for (int i = 0; i < mat[0].length; i++) {
            for (int j = 0; j < mat.length; j++) {
                transponierte[i][j]=mat[j][i];
            }
        }
        return transponierte;
    }

    public static double[][] berechneSumme(double[][] matA, double[][] matB)
throws FalscheMatrixAktionException{
    if ( matA.length != matB.length || matA[0].length != matB[0].length) {
        throw new FalscheMatrixAktionException();
    }

    double[][] summe=new double[matA.length][matA[0].length];
    for (int i = 0; i < matA.length; i++) {
        for (int j = 0; j < matA[0].length; j++) {
            summe[i][j]=matA[i][j]+matB[i][j];
        }
    }
    return summe;
}

    public static double[][] berechneDifferenz(double[][] matA, double[][] matB)
throws FalscheMatrixAktionException{
    if ( matA.length != matB.length || matA[0].length != matB[0].length) {
        throw new FalscheMatrixAktionException();
    }
    double[][] differenz=new double[matA.length][matA[0].length];

```

```

        for (int i = 0; i < matA.length; i++) {
            for (int j = 0; j < matA[0].length; j++) {
                differenz[i][j]=matA[i][j]-matB[i][j];
            }
        }
        return differenz;
    }

    public static double[][] berechneProdukt(double[][] matA, double[][] matB)
    throws FalscheMatrixAktionException{
        if (matA[0].length != matB.length) {
            throw new FalscheMatrixAktionException();
        }
        double[][] produkt=new double[matA.length][matB[0].length];
        for (int i = 0; i < matA.length; i++) {
            for (int j = 0; j < matB[0].length; j++) {
                double element=0;
                for(int k =0;k<matA[0].length;k++) {
                    element += matA[i][k]*matB[k][j];
                }
                produkt[i][j]=element;
            }
        }
        return produkt;
    }

    public static void main(String[] args) {

        double[][] matrixA = new double[2][2];
        double[][] matrixB = new double[2][2];
        // double[][] matrixErgebnis = new double[2][2];

        liesMatrixEin(matrixA);
        liesMatrixEin(matrixB);

        System.out.println("Determinante: "+berechneDeterminante(matrixA));
        System.out.println("Determinante: "+berechneDeterminante(matrixB));

        try {
            System.out.println("Spur: "+berechneSpur(matrixA));
        } catch (FalscheMatrixAktionException e) {
            e.printStackTrace();
        }
        try {
            System.out.println("Spur: "+berechneSpur(matrixB));
        } catch (FalscheMatrixAktionException e) {
            e.printStackTrace();
        }

        zeigeMatrix(bestimmeTransponierte(matrixA));
        zeigeMatrix(bestimmeTransponierte(matrixB));
    }

```

```

        try {
            zeigeMatrix(berechneInverse(matrixA));
            zeigeMatrix(berechneInverse(matrixB));
        } catch (KeineInverseException e) {
            e.printStackTrace();
        }

        try {
            zeigeMatrix(berechneSumme(matrixA,matrixB));
        } catch (FalscheMatrixAktionException e) {
            // TODO Auto-generated catch block
            e.printStackTrace();
        }

        try {
            zeigeMatrix(berechneDifferenz(matrixA,matrixB));
        } catch (FalscheMatrixAktionException e) {
            // TODO Auto-generated catch block
            e.printStackTrace();
        }

        try {
            zeigeMatrix(berechneProdukt(matrixA,matrixB));
        } catch (FalscheMatrixAktionException e) {
            // TODO Auto-generated catch block
            e.printStackTrace();
        }
    }

// public double[][] getRaw() {
// return data;
//}

// public void zeigeMatrix() {
// System.out.println("=====");
// for (int i = 0; i < data.length; i++) {
//     for (int j = 0; j < data[0].length; j++) {
//         System.out.printf("%8.2f", data[i][j]);
//     }
//     System.out.println();
// }
// System.out.println("=====");
//}
//
//public void liesMatrixEin() {
// System.out.println("=====");
// for (int i = 0; i < data.length; i++) {
//     for (int j = 0; j < data[0].length; j++) {
//         data[i][j] = Input.readDouble("Matrizelement(" + (i + 1) + " (" + (j + 1) + " +
//"): ");
//     }
// }

```

```

// }
// System.out.println("=====");
//}

//public double determinant() {
//  if (zeilen() != 2 || spalten() != 2) {
//    throw new IllegalArgumentException("Determinante aktuell nur fr 2x2");
//  }
//  double determinante = 0;
//  determinante = this.get(0, 0)*this.get(1, 1)-this.get(0, 1)*this.get(1, 0);
//  return determinante;
//}

//public Matrix inverse() throws KeineInverseException {
//  if (zeilen() != 2 || spalten() != 2) {
//    throw new IllegalArgumentException("Inverse aktuell nur fr 2x2");
//  }
//  Matrix inverse = new Matrix(zeilen(),spalten());
//  double det = determinant();
//  //
//  if (det == 0) {
//    throw new KeineInverseException();
//  }
//  //
//  inverse.set(0,0, get(1,1) / det);
//  inverse.set(0,1, -get(0,1) / det);
//  inverse.set(1,0,-get(1,0) / det);
//  inverse.set(1,1, get(0,0) / det);
//  return inverse;
//}

// public double determinant() {
//  if (this.zeilen() != this.spalten()) {
//    throw new IllegalArgumentException("Die Determinante ist nur fuer
quadratische Matrizen definiert.");
//  }
//  //
//  if (this.zeilen() == 1) {
//    return this.get(0, 0);
//  }
//  //
//  if (this.zeilen() == 2) {
//    return this.get(0, 0) * this.get(1, 1) - this.get(0, 1) * this.get(1, 0);
//  }
//  //
//  double summe = 0.0;
//  for (int j = 0; j < this.spalten(); j++) {
//    Matrix minor = this.streicheZeileUndSpalte(0, j);
//    double vorzeichen = (j % 2 == 0) ? 1 : -1;
//    summe += vorzeichen * this.get(0, j) * minor.determinant();
//  }
//  return summe;
//}
//

```

```

//public Matrix inverse() {
//  if (this.zeilen() != this.spalten()) {
//    throw new IllegalArgumentException("Die Inverse ist nur fuer quadratische
Matrizen definiert.");
//  }
//
//  double det = this.determinant();
//  if (det == 0) {
//    throw new IllegalArgumentException("Die Matrix ist nicht invertierbar, da
die Determinante 0 ist.");
//  }
//  if (zeilen() == 1) {
//    Matrix result = new Matrix(1,1);
//    result.set(0,0, 1.0 / get(0,0));
//    return result;
//  }
//
//  int n = this.zeilen();
//  Matrix kofaktoren = new Matrix(n, n);
//
//  for (int i = 0; i < n; i++) {
//    for (int j = 0; j < n; j++) {
//      Matrix minor = this.streicheZeileUndSpalte(i, j);
//      double vorzeichen = ((i + j) % 2 == 0) ? 1 : -1;
//      kofaktoren.set(i, j, vorzeichen * minor.determinant());
//    }
//  }
//
//  Matrix adjungierte = kofaktoren.transpose();
//  Matrix result = new Matrix(n, n);
//
//  for (int i = 0; i < n; i++) {
//    for (int j = 0; j < n; j++) {
//      result.set(i, j, adjungierte.get(i, j) / det);
//    }
//  }
//
//  return result;
//}
// public Matrix inverse() {
//  if (this.zeilen() != this.spalten()) {
//    throw new IllegalArgumentException("Die Inverse ist nur fuer
quadratische Matrizen definiert.");
//  }
//
//  double det = this.determinant();
//  if (det == 0) {
//    throw new IllegalArgumentException("Die Matrix ist nicht invertierbar,
da die Determinante 0 ist.");
//  }
//
//  if (this.zeilen() == 1) {
//    Matrix result = new Matrix(1, 1);
//    result.set(0, 0, 1.0 / this.get(0, 0));

```

```

//      return result;
//  }
//
//  int n = this.zeilen();
//  Matrix adjungierte = adjugate();
//  Matrix result = new Matrix(n, n);
//
//  for (int i = 0; i < n; i++) {
//      for (int j = 0; j < n; j++) {
//          result.set(i, j, adjungierte.get(i, j) / det);
//      }
//  }
//
//  return result;
//}
// public Matrix streicheZeileUndSpalte(int zeile, int spalte) {
//  Matrix result = new Matrix(this.zeilen() - 1, this.spalten() - 1);
//
//  int neueZeile = 0;
//  for (int i = 0; i < this.zeilen(); i++) {
//      if (i == zeile) {
//          continue;
//      }
//
//      int neueSpalte = 0;
//      for (int j = 0; j < this.spalten(); j++) {
//          if (j == spalte) {
//              continue;
//          }
//
//          result.set(neueZeile, neueSpalte, this.get(i, j));
//          neueSpalte++;
//      }
//      neueZeile++;
//  }
//
//  return result;
//}
//
//  return this.streicheZeileUndSpalte(zeileZuStreichen, spalteZuStreichen);
//
//  public static void main(String[] args) {
//  Matrix a = new Matrix(2,2);
//  a.set(0,0,1);
//  a.set(0,1,2);
//  a.set(1,0,-3);
//  a.set(1,1,4);
//
//  Matrix m = new Matrix(new double[][]{{5}});
//
//  System.out.println(a);
//  System.out.println("Det: " + a.determinant());
//  try {
//      System.out.println(a.inverse());

```



```

// } catch (IllegalArgumentException e) {
//     e.printStackTrace();
// }
// System.out.println(m);
// System.out.println("Det: " + m.determinant());
// try {
//     System.out.println(m.inverse());
// } catch (IllegalArgumentException e) {
//     e.printStackTrace();
// }
//
//
// System.out.println(a.multiply(a.inverse()));
//
// Matrix b= new Matrix(3,3);
// b.set(0, 0, 1);
// b.set(0, 1, 2);
// b.set(0, 2, 3);
//
// b.set(1, 0, 0);
// b.set(1, 1, 1);
// b.set(1, 2, 4);
//
// b.set(2, 0, 5);
// b.set(2, 1, 6);
// b.set(2, 2, 0);
//
// System.out.println(b);
// System.out.println("Det: " + b.determinant());
// try {
//     System.out.println(b.inverse());
// } catch (IllegalArgumentException e) {
//     e.printStackTrace();
// }
// System.out.println(b.multiply(b.inverse()));
//
//
//
//}

}

```

Package: Wrapperklassen

Klasse: Summenerzeugung

```

package Wrapperklassen;

public class Summenerzeugung {

```

```

public static void main(String[] args) {

    int summe=0;
    int counter=0;
    for (String item:args) {
        int summand=0;

        try {
            summand=Integer.parseInt(item);
            summe+=summand;
            System.out.println(++counter) + ". Summand: "+summand);
        } catch (NumberFormatException e) {
            System.out.println(e.getMessage());
        }

    }

    System.out.println("Die Summe der "+counter+" Summanden ist: "+summe);

}

}

```

Package: controller

Klasse: TaschenrechnerController

```

package controller;

import model.*;
import view.TaschenrechnerView;

public class TaschenrechnerController {

    private TaschenrechnerView view;
    private TaschenrechnerModel model;

    public TaschenrechnerController(
        TaschenrechnerView view,
        TaschenrechnerModel model) {

        this.view = view;
        this.model = model;

        initController();
    }

    private void initController() {

```

```

        view.btnPlus.addActionListener(
            e -> rechne(new PlusStrategy()));

        view.btnMinus.addActionListener(
            e -> rechne(new MinusStrategy()));

        view.btnMal.addActionListener(
            e -> rechne(new MalStrategy()));

        view.btnGeteilt.addActionListener(
            e -> rechne(new GeteiltStrategy()));
    }

    private void rechne(OperationStrategy strategy) {

        try {

            double a = Double.parseDouble(
                view.txt1.getText());

            double b = Double.parseDouble(
                view.txt2.getText());

            if(strategy instanceof GeteiltStrategy
                && b == 0) {

                view.txtErgebnis.setText(
                    "Division durch 0!");

                return;
            }

            double result =
                model.berechne(a,b,strategy);

            view.txtErgebnis.setText(
                String.valueOf(result));

        }

        catch(NumberFormatException ex) {

            view.txtErgebnis.setText(
                "Ungltige Eingabe");

        }
    }
}
package model;

public class TaschenrechnerModel {

    public double berechne(
        double a,
        double b,

```

```

        OperationStrategy strategy) {

            return strategy.berechne(a, b);
        }
    }package model;

    public class TaschenrechnerModel {

        public double berechne(
            double a,
            double b,
            OperationStrategy strategy) {

            return strategy.berechne(a, b);
        }
    }package view;

    import javax.swing.*.*;
    import java.awt.*.*;

    public class TaschenrechnerView extends JFrame {

        public JTextField txt1 = new JTextField();
        public JTextField txt2 = new JTextField();
        public JTextField txtErgebnis = new JTextField();

        public JButton btnPlus = new JButton("+");
        public JButton btnMinus = new JButton("-");
        public JButton btnMal = new JButton("x");
        public JButton btnGeteilt = new JButton("/");

        public TaschenrechnerView() {

            setTitle("MVC Taschenrechner");

            setSize(450,300);

            setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

            setLayout(null);

            txt1.setBounds(30,50,100,25);
            add(txt1);

            txt2.setBounds(30,100,100,25);
            add(txt2);

            txtErgebnis.setBounds(200,50,150,50);
            add(txtErgebnis);

            btnPlus.setBounds(280,150,80,25);
            add(btnPlus);

            btnMinus.setBounds(280,180,80,25);

```

```

        add(btnMinus);

        btnMal.setBounds(280,210,80,25);
        add(btnMal);

        btnGeteilt.setBounds(280,240,80,25);
        add(btnGeteilt);
    }
}package Timer;

import javax.swing.*.*;
import java.awt.*.*;

public class TimeInputDialog extends JDialog {

    private JTextField minutesField;
    private JTextField secondsField;
    private boolean confirmed = false;

    public TimeInputDialog(JFrame parent) {
        super(parent, "Neue Zeit", true);

        setSize(300, 180);
        setLocationRelativeTo(parent);
        setLayout(new BorderLayout(10, 10));

        // ICON oben links
        JLabel icon = new JLabel("?");
        icon.setOpaque(true);
        icon.setBackground(new Color(46, 125, 50)); // grn
        icon.setForeground(Color.BLACK);
        icon.setFont(new Font("SansSerif", Font.BOLD, 20));
        icon.setHorizontalAlignment(SwingConstants.CENTER);
        icon.setPreferredSize(new Dimension(40, 40));

        JPanel top = new JPanel(new FlowLayout(FlowLayout.LEFT));
        top.add(icon);

        add(top, BorderLayout.NORTH);

        JPanel center = new JPanel(new FlowLayout());

        minutesField = new JTextField(3);
        secondsField = new JTextField(3);

        center.add(new JLabel("Min:"));
        center.add(minutesField);
        center.add(new JLabel("Sek:"));
        center.add(secondsField);

        add(center, BorderLayout.CENTER);

        JPanel bottom = new JPanel();

```

```

        JButton ok = new JButton("OK");
        JButton cancel = new JButton("Abbrechen");

        ok.addActionListener(e -> {
            confirmed = true;
            dispose();
        });

        cancel.addActionListener(e -> dispose());

        bottom.add(ok);
        bottom.add(cancel);

        add(bottom, BorderLayout.SOUTH);
    }

    public boolean isConfirmed() {
        return confirmed;
    }

    public int getMinutes() {
        return Integer.parseInt(minutesField.getText().trim());
    }

    public int getSeconds() {
        return Integer.parseInt(secondsField.getText().trim());
    }
}

package Timer;
//
//import java.awt.EventQueue;
//
//import javax.swing.JFrame;
//import javax.swing.JPanel;
//import javax.swing.border.EmptyBorder;
//import javax.swing.JButton;
//import javax.swing.JToolBar;
//import javax.swing.JMenuBar;
//import javax.swing.JMenu;
//import javax.swing.JMenuItem;
//import java.awt.FlowLayout;
//import javax.swing.BoxLayout;
//import javax.swing.JLabel;
//import javax.swing.SwingConstants;
//import javax.swing.JProgressBar;
//import java.awt.GridLayout;
//import javax.swing.JTextField;
//import javax.swing.JPopupMenu;
//import java.awt.Component;
//import java.awt.event.MouseAdapter;
//import java.awt.event.MouseEvent;
//import java.awt.Dimension;
//import javax.swing.JTextArea;
//
//public class Timer_Gui extends JFrame {

```

```

//
// private static final long serialVersionUID = 1L;
// private JPanel contentPane;
// private JTextField txtMinuteneingabe;
// private JTextField txtSekundeneingabe;
// private JTextField txtBitteAnzahlMinuten;
//
// /**
//  * Launch the application.
//  */
// public static void main(String[] args) {
//     EventQueue.invokeLater(new Runnable() {
//         public void run() {
//             try {
//                 Timer_Gui frame = new Timer_Gui();
//                 frame.setVisible(true);
//             } catch (Exception e) {
//                 e.printStackTrace();
//             }
//         }
//     });
// }
//
// /**
//  * Create the frame.
//  */
// public Timer_Gui() {
//     setResizable(false);
//     setTitle("Count Down");
//     setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
//     setBounds(100, 100, 450, 300);
//
//     JMenuBar menuBar = new JMenuBar();
//     setJMenuBar(menuBar);
//
//     JMenu mnNewMenu = new JMenu("Timer");
//     menuBar.add(mnNewMenu);
//
//     JMenuItem mntmNewMenuItem = new JMenuItem("Neu...");
//     mnNewMenu.add(mntmNewMenuItem);
//
//     JMenuItem mntmNewMenuItem_1 = new JMenuItem("Starten");
//     mnNewMenu.add(mntmNewMenuItem_1);
//
//     JMenuItem mntmNewMenuItem_2 = new JMenuItem("Stoppen");
//     mnNewMenu.add(mntmNewMenuItem_2);
//
//     JMenuItem mntmNewMenuItem_3 = new JMenuItem("Reset");
//     mnNewMenu.add(mntmNewMenuItem_3);
//
//     JMenuItem mntmNewMenuItem_4 = new JMenuItem("Beenden");
//     mnNewMenu.add(mntmNewMenuItem_4);
//
//     JMenu mnNewMenu_1 = new JMenu("About");

```

```

//      menuBar.add(mnNewMenu_1);
//
//      JMenuItem mntmNewMenuItem_5 = new JMenuItem("gemacht vom Benny");
//      mnNewMenu_1.add(mntmNewMenuItem_5);
//      contentPane = new JPanel();
//      contentPane.setBorder(new EmptyBorder(5, 5, 5, 5));
//      setContentPane(contentPane);
//      contentPane.setLayout(new GridLayout(0, 6, 0, 0));
//
//      JProgressBar progressBar = new JProgressBar();
//      contentPane.add(progressBar);
//
//      JLabel lblNewLabel_1 = new JLabel("Minuten");
//      contentPane.add(lblNewLabel_1);
//
//      txtMinuteneingabe = new JTextField();
//      txtMinuteneingabe.setText("Minuten-Eingabe");
//      contentPane.add(txtMinuteneingabe);
//      txtMinuteneingabe.setColumns(10);
//
//      JTextArea textArea = new JTextArea();
//      textArea.setText(":");
//      contentPane.add(textArea);
//
//      JLabel lblNewLabel_2 = new JLabel("Sekunden");
//      contentPane.add(lblNewLabel_2);
//
//      txtSekundeneingabe = new JTextField();
//      txtSekundeneingabe.setText("Sekunden-Eingabe");
//      contentPane.add(txtSekundeneingabe);
//      txtSekundeneingabe.setColumns(10);
//
//      JButton btnNewButton = new JButton("Neu");
//      contentPane.add(btnNewButton);
//
//      JPopupMenu popupMenu = new JPopupMenu();
//      popupMenu.setPopupSize(new Dimension(300, 120));
//      addPopup(btnNewButton, popupMenu);
//
//      JLabel lblNewLabel_3 = new JLabel("Eingabe");
//      popupMenu.add(lblNewLabel_3);
//
//      JLabel lblNewLabel_4 = new JLabel("Bitte Anzahl Minuten < 100
eingeben...");
//      popupMenu.add(lblNewLabel_4);
//
//      txtBitteAnzahlMinuten = new JTextField();
//      txtBitteAnzahlMinuten.setText("Bitte Anzahl Minuten < 100 eingeben...");
//      popupMenu.add(txtBitteAnzahlMinuten);
//      txtBitteAnzahlMinuten.setColumns(10);
//
//      JButton btnNewButton_3 = new JButton("OK");
//      popupMenu.add(btnNewButton_3);
//

```



```

//      JButton btnNewButton_4 = new JButton("Abbrechen");
//      popupMenu.add(btnNewButton_4);
//
//      JButton btnNewButton_1 = new JButton("Start");
//      contentPane.add(btnNewButton_1);
//
//      JButton btnNewButton_2 = new JButton("Reset");
//      contentPane.add(btnNewButton_2);
//
//      JLabel lblNewLabel = new JLabel("Zeitanzeige");
//      lblNewLabel.setVerticalAlignment(SwingConstants.TOP);
//      lblNewLabel.setEnabled(false);
//      contentPane.add(lblNewLabel);
//
//  }
//
//  private static void addPopup(Component component, final JPopupMenu popup) {
//      component.addMouseListener(new MouseAdapter() {
//          public void mousePressed(MouseEvent e) {
//              if (e.isPopupTrigger()) {
//                  showMenu(e);
//              }
//          }
//          public void mouseReleased(MouseEvent e) {
//              if (e.isPopupTrigger()) {
//                  showMenu(e);
//              }
//          }
//          private void showMenu(MouseEvent e) {
//              popup.show(e.getComponent(), e.getX(), e.getY());
//          }
//      });
//  }
//}

```

Package: main

Klasse: Main

```

package main;

import controller.TaschenrechnerController;
import model.TaschenrechnerModel;
import view.TaschenrechnerView;

public class Main {

    public static void main(String[] args) {

        TaschenrechnerView view =
            new TaschenrechnerView();
    }
}

```

```

        TaschenrechnerModel model =
            new TaschenrechnerModel();

        new TaschenrechnerController(view, model);

        view.setVisible(true);
    }
}package Taschenrechner_GUI;

public class MalStrategy implements OperationStrategy {
    public double berechne(double a, double b) {
        return a * b;
    }
}package model;

public class MalStrategy implements OperationStrategy {

    @Override
    public double berechne(double a, double b) {
        return a * b;
    }
}package model;

public class MalStrategy implements OperationStrategy {

    @Override
    public double berechne(double a, double b) {
        return a * b;
    }
}package Matritzenrechner;

import IO.Input;

public class Matritzenrechner {

    public static void main(String[] args) {

        int zeilenanzahl;
        do {
            zeilenanzahl= Input.readInt("Geben Sie die Anzahl der Zeilen ein: ");
            if(zeilenanzahl<1) {
                System.out.println("Fehlerhafte Eingabe: "+zeilenanzahl);
            }
        } while (zeilenanzahl<1);

        int spaltenanzahl;
        do {
            spaltenanzahl= Input.readInt("Geben Sie die Anzahl der Spalten an: ");
            if (spaltenanzahl<1) {
                System.out.println("Fehlerhafte Eingabe: "+ spaltenanzahl);
            }
        } while (spaltenanzahl<1);
    }
}

```

```

int[][] originalmatrix = erzeugeMatrix(zeilenanzahl, spaltenanzahl);

int moeglichkeiten=3;
System.out.println("Welche Operation wollen Sie durchfhren?");
System.out.println("T:  Transponieren");
System.out.println("Z:  Zeilensumme");
System.out.println("S:  Spaltensumme");
if(spaltenanzahl==zeilenanzahl) {
    System.out.println("P:  Spurberechnung");
    moeglichkeiten++;
}
String operation=Input.readString("Whllen Sie eine der "+moeglichkeiten +
Operationen: ");

System.out.println("Originalmatzrix:");
zeigeMatrix(originalmatrix);
treffeAuswahl(operation, originalmatrix);

}

public static int[][] transponieren(int[][] a) {
    int[][] transponierte = new int[a[0].length][a.length];
    for (int i = 0; i < a[0].length; i++) {
        for (int j = 0; j < a.length; j++) {
            transponierte[i][j] = a[j][i];
        }
    }
    return transponierte;
}

public static int[] zeilensumme(int[][] a) {
    int[] zeilensumme = new int[a.length];
    for (int i = 0; i < a.length; i++) {
        for (int j = 0; j < a[0].length; j++) {
            zeilensumme[i] += a[i][j];
        }
    }
    return zeilensumme;
}

public static int[] spaltensumme(int[][] a) {
    int[] spaltensumme = new int[a[0].length];
    for (int j = 0; j < a[0].length; j++) {
        for (int i = 0; i < a.length; i++) {
            spaltensumme[j] += a[i][j];
        }
    }
    return spaltensumme;
}

public static int spur(int[][] a) {
    int spur = 0;

```

```

        for (int i = 0; i < a.length; i++) {
            spur += a[i][i];
        }
        return spur;
    }

    public static int[][] erzeugeMatrix(int zeilen, int spalten) {
        int[][] erzeugte = new int[zeilen][spalten];
        for (int i = 0; i < zeilen; i++) {
            for (int j = 0; j < spalten; j++) {
                erzeugte[i][j] = (int) (101 * Math.random());
            }
        }

        return erzeugte;
    }

    public static void zeigeMatrix(int[][] a) {
        for (int i = 0; i < a.length; i++) {
            for (int j = 0; j < a[0].length; j++) {
                System.out.print(a[i][j] + "\t");
            }
            System.out.println();
        }
    }

    public static void treffeAuswahl(String op, int[][] originalmatrix) {

        switch (op.toLowerCase()) {

            case "t": {
                System.out.println("Transponierte Matrix: ");
                zeigeMatrix(transponieren(originalmatrix));
                break;
            }
            case "z": {
                System.out.println("Zeilensumme: ");
                for (int item : zeilensumme(originalmatrix)) {
                    System.out.println(item);
                }
                break;
            }
            case "s": {
                System.out.println("Spaltensumme: ");
                for (int item : spaltensumme(originalmatrix)) {
                    System.out.print(item + "\t");
                }
                break;
            }
            case "p": {
                if(originalmatrix.length!= originalmatrix[0].length) {
                    System.out.println("Eine Spur ist nur fr quadratische Matritzen definiert!");
                }
                System.out.println("Fehlerhafte Eingabe: " + op);
            }
        }
    }

```

```

        System.out.println("Welche Operation wollen Sie durchfhren?");
        System.out.println("T:  Transponieren");
        System.out.println("Z:  Zeilensumme");
        System.out.println("S:  Spaltensumme");
        op=Input.readString("Whllen Sie eine der 3 Operationen: ");
        treffeAuswahl(op,orginalmatrix);
    }
    else {
        System.out.println("Spurberechnung: ");
        System.out.println("Die Spur ist: " + spur(orginalmatrix));
    }
    break;
}
default:
    System.out.println("Fehlerhafte Eingabe: " + op);
    System.out.println("Welche Operation wollen Sie durchfhren?");
    System.out.println("T:  Transponieren");
    System.out.println("Z:  Zeilensumme");
    System.out.println("S:  Spaltensumme");
    System.out.println("P:  Spurberechnung");
    op=Input.readString("Whllen Sie eine der 4 Operationen: ");
    treffeAuswahl(op,orginalmatrix);
}

}

}

```

Package: model

Klasse: OperationStrategy

```

package model;

public interface OperationStrategy {
    double berechne(double a, double b);
}package model;

public interface OperationStrategy {
    double berechne(double a, double b);
}package Pakrgebuehren;

import IO.Input;

public class Parkgebuehren {

    public static void main(String[] args) {

```

```

int stunden;
do {
    stunden = Input.readInt("Geben Sie die Anzahl der Stunden ein: ");
    if (stunden < 0) {
        System.out.println("Fehlerhafte Eingabe: " + stunden);
    }
} while (stunden < 0);
int minuten;
do {
    minuten = Input.readInt("Geben Sie die Anzahl der Minuten ein: ");
    if (minuten < 0)
        System.out.println("Fehlerhafte Eingabe: " + minuten);
} while (minuten < 0);
stunden += minuten / 60;
minuten = minuten % 60;
double gebuehren = 0;

if (stunden == 0) {
    gebuehren = 0.5 * ((minuten + 14) / 15);
} else if (stunden < 4 && stunden > 1 || stunden==4 && minuten==0 ) {
    gebuehren = 0.3 * ((stunden * 60 + minuten + 14) / 15);
} else if (stunden > 4 || stunden==4 && minuten > 0) {
    if (minuten == 0) {
        gebuehren = 0.75 * stunden;
    } else {
        gebuehren = 0.75 * (stunden + 1);
    }
} else {
    System.err.println("Ungltige Eingabe!");
}

System.out.printf("Die Parkgebühren fr %d Stunden und %d Minuten belaufen
sich auf: %.2f ",stunden, minuten, gebuehren);

}

}

```